

**MULTI-MODAL AERIAL SYSTEM FOR REAL-TIME SCENE
UNDERSTANDING, 3D RECONSTRUCTION, AND
GEOSPATIAL ANALYSIS**

Major Project Report

Submitted in partial fulfillment of the requirements for the award of the degree

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

by

**D V S S ROHITH
A V GAUTAM
B E V S CHANDU
D YASWANTH**

**22L31A0542
22L31A0501
23L35A0504
22L31A0534**

Under the Guidance of

Dr. N. Thirupathi Rao

(Professor)



VIGNAN'S **INSTITUTE OF INFORMATION TECHNOLOGY**
(AUTONOMOUS)

(Approved by AICTE-New Delhi & Affiliated to JNTUGV, Vizianagaram)
Beside VSEZ, Duvvada, Vadiapudi Post, Gajuwaka, Visakhapatnam - 530 049.

**Department of Computer Science & Engineering
2025-2026**



CERTIFICATE

This is to certify that the project report entitled “**MULTI-MODAL AERIAL SYSTEM FOR REAL-TIME SCENE UNDERSTANDING, 3D RECONSTRUCTION, AND GEOSPATIAL ANALYSIS**” is a bonafide record of project work carried out under my supervision by **D V S S Rohith(22L31A0542)**, **A V Gautam(22L31A0501)**, **B E V S Chnadu(23L35A0504)**, **D Yaswanth(22L31A0534)** during the academic year 2022-2026, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering of VIGNAN’S INSTITUTE OF INFORMATION TECHNOLOGY (Autonomous). The results embodied in this major project report have not been submitted to any other University for the award of any Degree.

Signature of Project Guide

Dr. N. Thirupathi Rao
Professor

Head of the Department

Mr. B. Dinesh Reddy
Associate Professor

Signature of External Examiner

DECLARATION

We hereby declare that the project report entitled “**MULTI-MODAL AERIAL SYSTEM FOR REAL-TIME SCENE UNDERSTANDING, 3D RECONSTRUCTION, AND GEOSPATIAL ANALYSIS**” has been written by us and not been submitted either in part or whole for the award of any degree.

D V S S Rohith	22L31A0542
A V Gautam	22L31A0501
B E V S Chandu	23L35A0504
D Yaswanth	22L31A0534

Date:

Place: Visakhapatnam

ACKNOWLEDGMENT

We consider it as a privilege to thank all those people who helped us a lot for the successful completion of the project “**MULTI-MODAL AERIAL SYSTEM FOR REAL-TIME SCENE UNDERSTANDING, 3D RECONSTRUCTION, AND GEOSPATIAL ANALYSIS**”.

First of all we would like to thank our project guide **Dr. N. Thirupathi Rao, Professor, Department of Computer Science & Engineering** for helping us a lot in completing our project work and for enlightening us with constructive suggestions for solving our problems patiently and helping us to improve the quality of work.

We would like to thank our ever-inspiring **Head of the Department, Computer Science and Engineering, Mr. B. Dinesh Reddy**, for his spontaneous response to every request though he is busy with his hectic schedule of administration and teaching.

We would like to thank the **Principal** of Vignan’s Institute of Information Technology **Dr. J. Sudhakar**, for his encouragement to us during the course of this project work.

We also express our sincere thanks to our beloved **Chairman** of Vignan’s Institutions **Dr. L. Rathaiah**, who has given us a lot of support and freedom during our project work.

We express our thanks to all **teaching faculty** of Department of Computer Science and Engineering, VIIT whose suggestions during reviews helped us in accomplishment of our project. We would like to thank all **non-teaching staff** of the Department of Computer Science and Engineering, VIIT for providing great assistance in accomplishment of our project.

D V S S Rohith	22L31A0542
A V Gautam	22L31A0501
B E V S Chandu	23L35A0504
D Yaswanth	22L31A0534

ABSTRACT

Unmanned aerial perception systems relying on a single sensing modality often degrade under low illumination, atmospheric disturbances, and occlusions. Additionally, conventional UAV pipelines decouple image enhancement from downstream perception, reducing reliability in time-critical operations. This work presents a multi-modal aerial system for real-time scene understanding, 3D reconstruction, and geospatial analysis through tightly integrated processing of synchronized RGB, thermal, and depth data. Architecture employs modality-specific encoders and cross-attention transformer-based fusion to reconstruct high-quality representations while preserving structural and radiometric consistency. These representations are directly coupled with perception modules for object detection, multi-object tracking, and spatial measurement, forming an end-to-end pipeline suitable for onboard deployment. This unified design eliminates disjoint stages, enabling simultaneous visual restoration and metric scene interpretation. Evaluation on UAVScenes, FLIR, VisDrone2019, and a custom RGB–thermal–depth dataset demonstrates robust performance under degraded conditions. The reconstruction module achieves 29.0 dB PSNR and 0.875 SSIM, improving PSNR by up to 8.1 dB over RGB-only baselines while reducing LPIPS to 0.181. Fusion-driven perception achieves 37.5 mAP (61.2 AP50), with consistent improvements for small object detection. Multi-object tracking reaches 61.7 MOTA and 69.8 IDF1, reducing identity switches by 19.3%. Metric reconstruction enables real-world measurement with height and distance errors below 5% and 3%. Optimized with ONNX and TensorRT, the system operates at 12–30 FPS on NVIDIA Jetson devices, achieving up to $4.1\times$ acceleration with minimal accuracy loss. Field deployment validates stable onboard performance and improved visibility in low-contrast environments.

Keywords: Aerial surveillance, real-time object detection, multi-spectral imaging, 3D reconstruction, autonomous UAVs, disaster response, geospatial intelligence.

TABLE OF CONTENTS

Name of the Chapter	Page No
---------------------	---------

Chapter-1 INTRODUCTION

1.1	Introduction	2
1.2	TRANSFORMER NETWORKS	3
1.3	Multi-Modal Data Fusion	4
1.4	Objectives of the Project	5
1.5	Problem Statement	6

Chapter-2 LITERATURE REVIEW

2.1	Review of Literature on UAV-Based Scene Understanding Systems	9
2.2	Multi-Modal Data Acquisition and Sensor Technologies	10
2.3	Challenges in Aerial Image Processing and Environmental Conditions	11
2.4	Deep Learning Approaches for Image Reconstruction	13
2.5	Recent Advancements in UAV-Based Vision Systems	14
2.6	Research Gaps	15
2.7	Motivation	15
2.8	Problem Statement and Research Objectives	16
2.9	Summary of Literature Review	16

Chapter-3 METHODOLOGY

3.1	Research Modal System Architecture	18
3.2	Data Collection and Preprocessing	19
3.3	Transformer-Based Multi-Modal Fusion and Image Reconstruction	24
3.4	Analytical Intelligence Layer	25
3.5	Deployment and Optimization	23

Chapter-4 RESULTS AND DISCUSSIONS

4.1	Reconstruction Quality Assessment	27
4.2	Object Detection Performance	29
4.3	Multi-Object Tracking Results	31
4.4	Metric Measurement Accuracy	33
4.5	Edge Deployment Performance	34
4.6	Ablation Studies	34
4.7	Qualitative Results	36
4.8	Real-Time System Demonstration	37
4.9	Advantages of Transformer-Based Fusion	39
4.10	Multi-Modal Complementarity	40
4.11	Real-Time Performance Trade-offs	40
4.12	Deployment Insights	41
4.13	Limitations and Future Work	41
4.14	Broader Impact	42

Chapter-5 CONCLUSION AND FUTURE WORK

5.1	Conclusion	44
5.2	Key Contributions of the Proposed System	45
5.3	Practical Implications and Applications	43
5.4	Future Scope and Research Directions	46

Chapter-6 References

References	48
------------	----

Appendix

1. Algorithm Code	50
2. Submission in IEEE conference Acknowledgement	70
3. Draft of the research paper	71

LIST OF FIGURES

S.No	Name of the Figure	Page No
1	Transformer Networks Architecture	4
2	Multi-Modal Sensor Fusion Framework	5
3	System Architecture Overview	18
4	Multi-Modal Data Preprocessing Pipeline	21
5	Scaling in Data Augmentation	22
6	Rotation in Data Augmentation	23
7	Flipping in Data Augmentation	23
8	Real-Time Adaptation Process	25
9	PSNR (dB) Across Degradation Conditions	27
10	SSIM Comparison Across Methods and Conditions	28
11	Object Detection Performance Metrics	29
12	Speed-Accuracy Trade-off in Object Detection	29
13	ROC Curves - Object Detection Methods	30
14	Multi-Object Tracking Performance	31
15	Tracking Errors: False Positives, False Negatives, ID Switches	31
16	ROC Curves - Multi-Object Tracking Methods	32
17	Metric Measurement Accuracy by Object Class	33
18	Ablation Study: Effect of Modalities on Performance	34
19	Fusion Strategy: Quality vs Model Complexity	35
20	3D Detection Performance Landscape	36
21	Real time System Demonstration	37
22	Input Modalities (Image/Stream)	38
23	Multi Object Detection Labeling	39
24	Object Detection and Labeling in Thermal Sensor Images	40

LIST OF TABLES

S.No	Name of the Table	Page No
1	Multi-Modal Data Sources and Description	20
2	Reconstruction Performance Under Degraded Conditions (PSNR, SSIM, LPIPS)	27
3	Object Detection Performance on the VisDrone2019 Test Set	30
4	Multi-Object Tracking Performance on the MOT-UAV Benchmark	32
5	Metric Measurement Accuracy Across Object Classes	33
6	Ablation Study on Modality Contribution and Fusion Strategy	34
7	Fusion Strategy Comparison for Reconstruction Quality and Model Complexity	35

ABBREVIATIONS

UAV	Unmanned Aerial Vehicle
RGB	Red, Green, Blue
CNN	Convolutional Neural Network
DL	Deep Learning
ML	Machine Learning
ANN	Artificial Neural Network
ViT	Vision Transformer
Swin	Shifted Window Transformer
MHA	Multi-Head Attention
MSA	Multi-Head Self-Attention
FFN	Feed Forward Network
PE	Positional Encoding
LN	Layer Normalization
MMDF	Multi-Modal Data Fusion
DNN	Deep Neural Network
FPN	Feature Pyramid Network
RPN	Region Proposal Network
IoU	Intersection over Union
mAP	Mean Average Precision
AP	Average Precision
ReLU	Rectified Linear Unit
GELU	Gaussian Error Linear Unit
SGD	Stochastic Gradient Descent
Adam / AdamW	Adaptive Moment Estimation (with Weight Decay)
PSNR	Peak Signal-to-Noise Ratio
SSIM	Structural Similarity Index Measure
LPIPS	Learned Perceptual Image Patch Similarity
FPS	Frames Per Second
MOTA	Multiple Object Tracking Accuracy
IDF1	Identification F1 Score
FP	False Positives

FN	False Negatives
IDSW	Identity Switches
YOLO	You Only Look Once
DETR	Detection Transformer
ONNX	Open Neural Network Exchange
IMU	Inertial Measurement Unit
GPS	Global Positioning System
LiDAR	Light Detection and Ranging
GIS	Geographic Information System
SLAM	Simultaneous Localization and Mapping
SAR	Synthetic Aperture Radar
FP32	32-bit Floating Point
FP16	16-bit Floating Point
INT8	8-bit Integer
SNR	Signal-to-Noise Ratio
UI	User Interface
ReconMMT	Reconstruction using a Multi-Modal Transformer for real-time geospatial scene understanding and sensing.

Chapter 1

Introduction

1.1 Introduction

Unmanned Aerial Vehicles (UAVs), commonly known as drones, are increasingly used in applications such as surveillance, disaster management, environmental monitoring, and precision agriculture. With the integration of artificial intelligence, UAVs are evolving into intelligent systems capable of real-time scene understanding and decision-making. A major challenge in aerial vision systems is the limitation of traditional RGB cameras, which often fail under adverse conditions such as low light, fog, smoke, and occlusions. These limitations reduce the reliability of perception, especially in critical scenarios like search-and-rescue operations. To overcome this, multi-modal sensing has emerged as an effective solution, where data from different sensors-such as RGB, thermal, and depth-is combined. RGB images provide color and texture information, thermal images capture heat signatures, and depth sensors provide spatial and geometric details. This project presents a multi-modal UAV-based system for real-time scene understanding and image reconstruction. The system integrates data from multiple sensors and uses advanced deep learning techniques to generate enhanced visual representations from degraded inputs. These reconstructed outputs are further used for tasks such as object detection, segmentation, tracking, and real-world measurement estimation.

A key feature of the proposed system is its ability to perform end-to-end processing, from data acquisition to intelligent analysis, in real time. The system is designed for deployment on edge devices, enabling efficient operation on UAV platforms with limited computational resources. By combining multi-modal fusion with intelligent analytics, the project aims to improve the robustness and accuracy of aerial perception systems in real-world environments. In this project, a multi-modal aerial system for real-time scene understanding and 3D reconstruction is developed. The reconstructed outputs are further processed to perform high-level analytical tasks such as object detection, segmentation, multi-object tracking, and real-world measurement estimation. Overall, this project aims

to bridge the gap between raw multi-sensor data and actionable intelligence by combining advancements in UAV technology, multi-modal data fusion, and deep learning-based computer vision, thereby contributing to the development of next-generation autonomous aerial systems.

1.2 TRANSFORMER NETWORKS

Transformer Networks are a class of deep learning models designed to process data using attention mechanisms. Unlike Convolutional Neural Networks (CNNs), which focus on local feature extraction, transformers capture global relationships and dependencies within the input. They are highly effective for complex tasks such as image reconstruction and multi-modal data fusion.

Transformer Architecture:

- **Self-Attention Mechanism:** Identifies important regions in the input by assigning different weights to different parts of the data.
- **Multi-Head Attention:** Allows the model to focus on multiple aspects of the input simultaneously for better feature representation.
- **Feed Forward Network (FFN):** Processes the attended features and enhances their representation.
- **Positional Encoding:** Adds spatial or sequential information to the input since transformers do not inherently understand order.
- **Cross-Attention:** Enables interaction between different data modalities such as RGB, thermal, and depth.

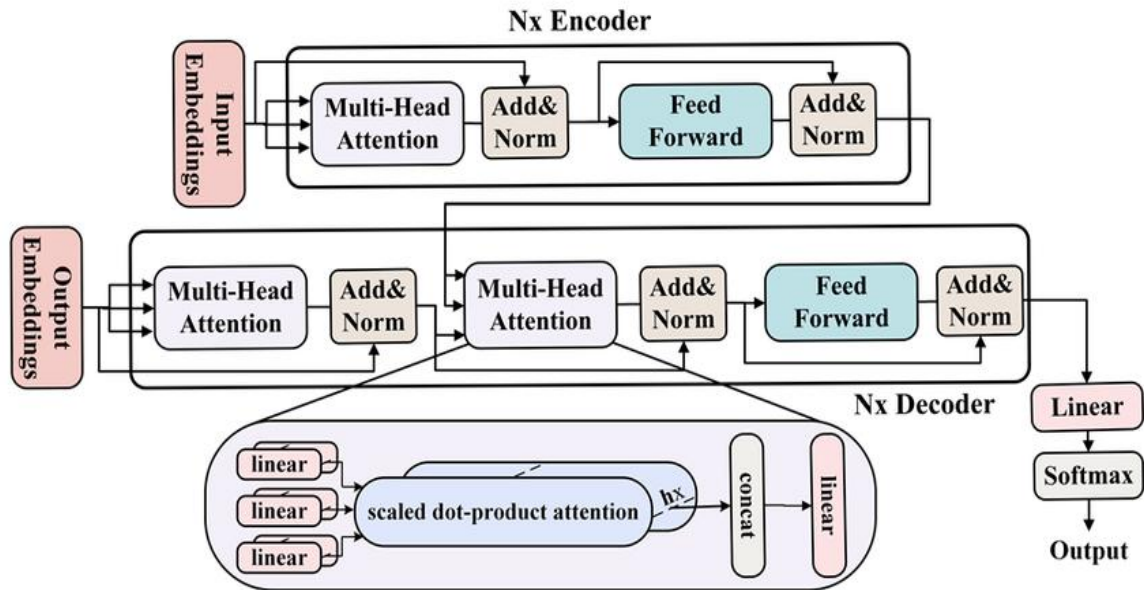


Figure 1: Transformer Networks Architecture

1.3 Multi-Modal Data Fusion

Multi-Modal Data Fusion is a technique in which information from multiple data sources or sensors is combined to produce more accurate and reliable results. In computer vision applications, relying on a single modality such as RGB images often leads to performance degradation under challenging conditions like low illumination, fog, smoke, and occlusions. To overcome these limitations, integrating complementary modalities such as thermal and depth data has become essential.

Multi-Modal Fusion Components:

- **RGB Data:** Provides color and texture information, useful for identifying object appearance and visual details.
- **Thermal Data:** Captures heat signatures, enabling detection in low-light or visually obstructed environments.
- **Depth Data:** Provides spatial and geometric information, helping in distance estimation and 3D understanding of the scene.
- **Feature-Level Fusion:** Combines features extracted from different modalities to create a unified representation.
- **Attention-Based Fusion:** Uses transformer-based attention mechanisms to learn the importance of each modality dynamically.

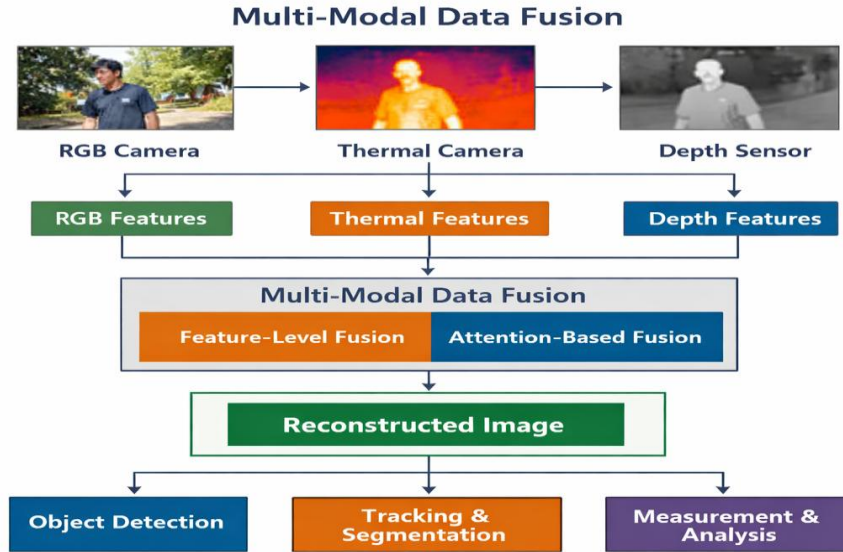


Figure 2: Multi-Modal Sensor Fusion Framework

1.4 Objectives of the Project

This project aims to develop an advanced multi-modal system that integrates deep learning with aerial sensing technologies for robust scene understanding and image reconstruction. The key objectives are:

1. To design and implement a UAV-based system capable of capturing multi-modal data including RGB, thermal, and depth information.
2. To develop a transformer-based deep learning model for effective fusion of multi-modal data and enhanced feature representation.
3. To reconstruct high-quality images by integrating complementary information from different sensor modalities.
4. To improve scene understanding through accurate detection, segmentation, and analysis of objects in diverse environmental conditions.
5. To ensure reliable performance in challenging scenarios such as low-light, fog, and occlusions using multi-modal inputs.

1.5 Problem Statement

Traditional UAV-based vision systems are limited by their dependence on single-modal data such as RGB images, which fail to perform effectively under challenging environmental conditions. This leads to inaccurate scene understanding and unreliable image reconstruction in real-world applications.

Key problems include:

- Difficulty in detecting and analyzing objects in low-light, foggy, or occluded environments.
- Limited performance of single-modal systems due to lack of complementary information.
- Inaccurate depth perception and poor spatial understanding of complex scenes.
- Inefficiency of conventional deep learning models in capturing long-range dependencies in visual data.

This project proposes a transformer-based multi-modal data fusion system that integrates RGB, thermal, and depth information to achieve accurate, reliable, and real-time scene understanding and image reconstruction.

Chapter 2

Literature Review

2.1 Review of Literature on UAV-Based Scene Understanding Systems

UAV-based scene understanding systems have evolved significantly with the advancement of computer vision and deep learning technologies. Early approaches relied on traditional image processing techniques, which were limited in handling complex environments and dynamic conditions. With the introduction of deep learning, especially Convolutional Neural Networks (CNNs), the ability to extract meaningful features from aerial imagery improved considerably. However, these methods primarily focused on single-modal data and struggled in challenging scenarios such as low-light conditions, occlusions, and adverse weather.

- **Deep Learning-Based UAV Vision Systems:** Recent studies highlight the use of CNN-based architectures for tasks such as object detection, image segmentation, and scene classification in UAV imagery. Models like Faster R-CNN, YOLO, and U-Net have been widely used for real-time aerial analysis. These models have shown strong performance in controlled environments but often fail to capture long-range dependencies and global context, which are crucial for accurate scene understanding in complex aerial views.
- **Transformer-Based Vision Models:** With the emergence of transformer architectures, such as Vision Transformer (ViT) and Swin Transformer, significant improvements have been achieved in capturing global relationships within images. Transformers utilize self-attention mechanisms to model long-range dependencies, making them highly effective for tasks like image reconstruction and multi-modal learning. Recent research indicates that transformer-based models outperform traditional CNNs in handling large-scale and complex visual data.
- **Multi-Modal Data Fusion Approaches:** To overcome the limitations of single-modal systems, researchers have explored multi-modal data fusion techniques that integrate RGB, thermal, and depth data. Studies show that combining complementary modalities enhances system robustness and improves performance

in challenging environments. Feature-level and attention-based fusion methods, particularly those using transformer networks, have demonstrated superior results in scene understanding and object detection tasks.

- **UAV Applications and Real-Time Challenges:** UAV-based systems are widely used in applications such as surveillance, disaster management, environmental monitoring, and smart agriculture. However, real-time processing, computational efficiency, and adaptability to varying environmental conditions remain major challenges. Research is increasingly focused on developing efficient architectures that balance accuracy and speed while maintaining robustness.
- **Challenges and Future Directions:** Despite significant advancements, UAV-based scene understanding systems still face challenges such as high computational requirements, data heterogeneity, and real-time deployment constraints. Future research is directed towards optimizing transformer models, improving multi-modal fusion strategies, and developing scalable solutions for real-world applications.

2.2 Multi-Modal Data Acquisition and Sensor Technologies

Multi-modal data acquisition plays a critical role in enhancing the performance of UAV-based scene understanding systems, as different sensing technologies provide complementary information about the environment. Relying on a single data modality often leads to incomplete or inaccurate interpretations, especially under challenging conditions such as low light, occlusions, and adverse weather. Therefore, integrating multiple sensors enables more robust and reliable perception in real-world applications.

- **RGB Imaging Sensors:** RGB cameras are widely used in UAV systems for capturing high-resolution visual data, including color, texture, and object details. They are effective in well-lit environments and are essential for tasks such as object detection and classification. However, their performance degrades significantly in poor lighting conditions, shadows, and visually obstructed environments, limiting their reliability in critical applications.
- **Thermal Imaging Sensors:** Thermal sensors capture infrared radiation emitted by objects, allowing the system to detect heat signatures regardless of lighting

conditions. This makes them highly useful for applications such as night-time surveillance, search and rescue operations, and environmental monitoring. Studies show that thermal imaging improves detection accuracy in scenarios where RGB data fails, particularly in foggy or smoke-filled environments.

- **Depth Sensing Technologies:** Depth sensors, including LiDAR and stereo vision systems, provide spatial and geometric information about the scene. This enables accurate distance estimation, object localization, and 3D reconstruction. Depth data is crucial for understanding scene structure and improving navigation and obstacle avoidance in UAV systems.
- **Sensor Fusion and Integration:** The integration of RGB, thermal, and depth sensors allows UAV systems to leverage the strengths of each modality while compensating for their individual limitations. Multi-modal data acquisition ensures that critical information is preserved even when one modality is unreliable. Advanced synchronization and calibration techniques are required to align data from different sensors for effective processing.
- **Role of Advanced Learning Models:** Recent advancements in deep learning, particularly transformer-based architectures, have enabled efficient processing and fusion of multi-modal data. These models can dynamically learn the importance of each modality and extract meaningful features for improved scene understanding and image reconstruction.

2.3 Challenges in Aerial Image Processing and Environmental Conditions

UAV-based aerial image processing systems face several challenges due to environmental variability and limitations in sensing technologies. These challenges significantly impact the accuracy of scene understanding, object detection, and image reconstruction. Addressing these issues is essential for developing robust and reliable multi-modal vision systems.

- **Low-Light and Illumination Variations:** Aerial imaging systems often operate in varying lighting conditions, including night-time or shadowed regions. RGB-based models struggle to capture meaningful features in such scenarios, leading to poor

detection and classification performance. Illumination inconsistencies can also introduce noise and reduce image clarity.

- **Adverse Weather Conditions:** Environmental factors such as fog, rain, smoke, and dust can degrade image quality and obscure important visual information. These conditions reduce visibility and affect the performance of traditional vision models, making it difficult to accurately interpret aerial scenes.
- **Occlusions and Complex Backgrounds:** Objects in aerial images are frequently partially hidden by obstacles such as trees, buildings, or terrain features. Additionally, complex and cluttered backgrounds make it challenging to distinguish objects of interest, reducing detection accuracy and increasing false predictions.
- **Limited Depth and Spatial Understanding:** Single-modal imaging systems lack sufficient spatial information, making it difficult to estimate distances and understand 3D scene structure. This limitation affects tasks such as navigation, obstacle avoidance, and accurate scene reconstruction.
- **High Computational Complexity:** Processing high-resolution aerial data in real-time requires significant computational resources. Advanced models, especially transformer-based architectures, demand high memory and processing power, which can be challenging for deployment on UAV platforms with limited resources.
- **Data Heterogeneity and Synchronization Issues:** In multi-modal systems, data from different sensors such as RGB, thermal, and depth must be synchronized and aligned accurately. Variations in resolution, frame rates, and sensor noise introduce complexities in data fusion and can affect overall system performance.
- **Scale and Resolution Variability:** Objects in aerial images often appear at different scales and resolutions due to varying flight altitudes and camera angles. This variation makes it difficult for models to consistently detect and analyze objects, affecting overall accuracy and performance.

2.4 Deep Learning Approaches for Image Reconstruction

Advancements in deep learning have significantly improved image reconstruction techniques, enabling high-quality visual output from complex and incomplete data. These approaches play a crucial role in UAV-based systems, where captured data is often affected by environmental conditions, noise, and sensor limitations. Deep learning models can learn complex patterns and restore missing or degraded information, making them essential for accurate scene understanding.

- **Autoencoders and Encoder-Decoder Models:** Autoencoders are designed to learn compact representations of input data and reconstruct images from encoded features. Encoder-decoder architectures, such as U-Net, are particularly effective in preserving spatial information and are commonly used in image restoration and segmentation tasks. These models help in reconstructing images from partially corrupt or incomplete inputs.
- **Transformer-Based Models:** Recent approaches utilize transformer architectures for image reconstruction due to their ability to capture global dependencies using attention mechanisms. Models such as Vision Transformer (ViT) and Swin Transformer have shown superior performance in reconstructing images by effectively integrating contextual information across the entire image.
- **Multi-Modal Reconstruction Techniques:** Combining data from multiple modalities such as RGB, thermal, and depth improves reconstruction quality. Multi-modal approaches leverage complementary information to recover missing details and enhance robustness. Transformer-based fusion methods further improve reconstruction by dynamically weighing the importance of each modality.

2.5 Recent Advancements in UAV-Based Vision Systems

Recent advancements in computer vision and aerial technologies have significantly enhanced the capabilities of UAV-based vision systems. These developments focus on improving accuracy, efficiency, and robustness in real-time scene understanding and image reconstruction, especially under challenging environmental conditions.

- **Transformer-Based Vision Models:** The introduction of transformer architectures such as Vision Transformer (ViT) and Swin Transformer has improved the ability to capture global context and long-range dependencies in images. These models outperform traditional CNNs in complex visual tasks and are widely used for high-precision image analysis and reconstruction.
- **Multi-Modal Learning Techniques:** Modern UAV systems increasingly integrate multiple data modalities such as RGB, thermal, and depth. Advanced fusion techniques, particularly attention-based methods, allow models to combine complementary information effectively, resulting in improved performance in low-light and occluded environments.
- **Edge Computing and Real-Time Processing:** Recent advancements in edge computing enable UAVs to process data locally, reducing latency and dependency on cloud infrastructure. Lightweight deep learning models and optimized architectures allow real-time decision-making, which is critical for applications like surveillance and disaster response.
- **3D Reconstruction and Spatial Understanding:** Techniques such as stereo vision, LiDAR integration, and deep learning-based depth estimation have enhanced 3D scene reconstruction capabilities. These methods enable accurate spatial analysis, object localization, and environment mapping from aerial data.
- **AI-Driven Automation in UAV Systems:** Artificial intelligence is enabling autonomous UAV operations, including navigation, obstacle avoidance, and target detection. Integration of deep learning models allows UAVs to adapt to dynamic environments and perform complex tasks with minimal human intervention.

2.6 Research Gaps

Despite significant advancements in UAV-based vision systems, several limitations still exist in achieving robust and reliable scene understanding. Current research highlights the need for improved integration of multi-modal data and efficient deep learning models that can perform accurately in real-world conditions.

- **Limited Effectiveness of Single-Modal Systems:** Most existing approaches rely heavily on RGB data, which performs poorly under challenging conditions such as low light, fog, and occlusions, leading to reduced accuracy.
- **Inefficient Multi-Modal Fusion Techniques:** Although multi-modal data fusion has been explored, many methods fail to effectively combine information from different sensors, resulting in suboptimal feature representation and performance.
- **High Computational Complexity of Advanced Models:** Transformer-based architecture provides better accuracy but require high computational resources, making real-time implementation on UAV systems challenging.

2.7 Motivation

The increasing demand for accurate and reliable aerial scene understanding in applications such as surveillance, disaster management, and environmental monitoring highlights the need for advanced vision systems. Traditional UAV-based approaches using single-modal data are insufficient to handle real-world challenges like poor lighting, occlusions, and adverse weather conditions.

- **Need for Robust Multi-Modal Systems:** To overcome the limitations of single-modal approaches, integrating multiple data sources such as RGB, thermal, and depth is essential for improving accuracy and reliability.
- **Advancements in Transformer-Based Learning:** Recent developments in transformer architectures provide an opportunity to capture global context and enhance feature representation, motivating their use in complex vision tasks.

2.8 Problem Statement and Research Objectives

This project aims to address the challenges in UAV-based scene understanding by leveraging advanced deep learning techniques and multi-modal data integration. Traditional aerial vision systems face limitations in accuracy and reliability under complex environmental conditions, which necessitates the development of more robust and intelligent solutions. The main objectives are:

1. To develop a UAV-based system capable of capturing and processing multi-modal data for enhanced scene understanding.
2. To design a transformer-based deep learning model for effective fusion of multi-modal inputs such as RGB, thermal, and depth data.
3. To improve image reconstruction and object detection accuracy under challenging conditions like low light, fog, and occlusions.
4. To enable real-time processing and efficient deployment of the system for applications such as surveillance and disaster management.

2.9 Summary of Literature Review

The literature review has highlighted the evolution of wildlife conservation model, emphasizing the importance of geographic factors, technological advancements, and community engagement in shaping effective strategies. Integrating AI, IoT, and eDNA analysis with traditional conservation practices has shown considerable promise in enhancing monitoring, protecting endangered species, and restoring habitats. However, research gaps in data integration, AI applications, and socio-political factors must be addressed to further improve conservation efforts globally.

Chapter 3

Methodology

Chapter – 3

METHODOLOGY

3.1 Research Model and System Architecture

This research adopts a system-oriented, data-driven approach to develop a multi-modal UAV-based intelligent vision system for real-time scene understanding, 3D reconstruction, and geospatial analysis. The design focuses on building an end-to-end pipeline that integrates multi-sensor data acquisition, preprocessing, transformer-based fusion, analytical intelligence, and deployment into a unified framework.

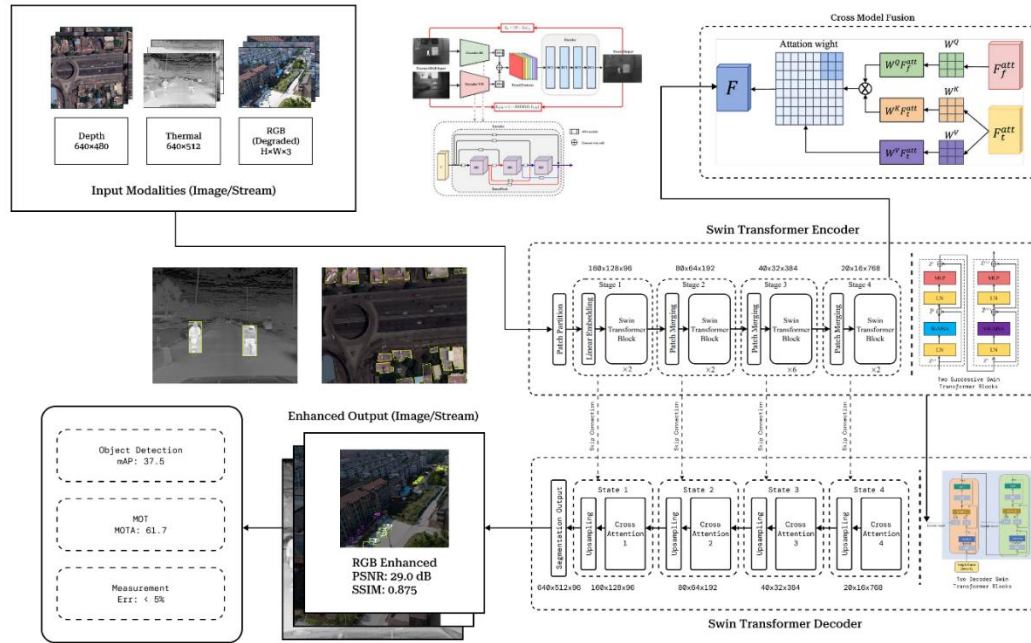


Figure 3: System Architecture Overview

The proposed system is structured as a modular architecture, where each component—data acquisition, preprocessing, reconstruction, analytics, and deployment—operates independently while maintaining seamless integration. This modularity ensures scalability, efficient computation, and adaptability to real-world UAV constraints.

The architecture consists of the following core layers:

- **Data Acquisition Layer:** Captures synchronized multi-modal inputs (RGB, thermal,

depth, GPS, IMU) from UAV platforms or datasets.

- **Preprocessing Layer:** Handles calibration, synchronization, normalization, and alignment of multi-modal data.
- **Image Reconstruction Module:** A transformer-based fusion architecture that reconstructs high-quality RGB representations from degraded multi-modal inputs.
- **Analytical Intelligence Layer:** Performs object detection, segmentation, multi-object tracking, and spatial measurement.
- **Visualization and UI Layer:** Provides real-time monitoring and offline analysis interfaces.
- **Deployment Layer:** Optimized for edge inference using ONNX and TensorRT for real-time UAV execution.

The system is designed for real-time performance on resource-constrained UAV platforms, ensuring robustness under challenging conditions such as low illumination, fog, occlusion, and motion disturbances.

3.2 Data Collection and Preprocessing

Data plays a crucial role in enabling accurate scene understanding and 3D reconstruction in UAV-based systems. In this project, **multi-modal data** is collected from diverse onboard and external sensing sources, each contributing complementary information about the environment.

The system integrates **visual, thermal, and spatial data streams** to enhance perception under varying environmental conditions such as low light, occlusion, and dynamic scenes.

The primary data sources used in this research include:

- **RGB Camera (UAV-Mounted):** High-resolution aerial images and video streams for visual scene understanding and object detection.
- **Thermal Imaging Sensors:** Infrared data used for detecting heat signatures, especially useful in low-light or obscured environments.
- **Depth Sensors / LiDAR:** Provides spatial and geometric information required for

accurate 3D reconstruction.

- **IMU (Inertial Measurement Unit):** Captures motion data such as acceleration and orientation, assisting in stabilization and trajectory estimation.
- **GPS Module:** Provides geolocation data for mapping, tracking, and geo-referenced reconstruction.
- **Public Datasets:** Benchmark datasets such as aerial imagery datasets (e.g., UAV123, KITTI) used for model training and validation.

Table 1: Multi-Modal Data Sources and Description

Data Source	Description	Format
RGB Camera Data	High-resolution aerial images and real-time video streams	Images, Videos
Thermal Sensor Data	Infrared images capturing temperature variations	Thermal Images
Depth / LiDAR Data	3D spatial information for reconstruction	Point Clouds, Depth Maps
GPS Data	Location coordinates for mapping and tracking	Geo-coordinates (CSV)
Public Datasets	Pre-collected datasets for training and benchmarking	Images, Annotations

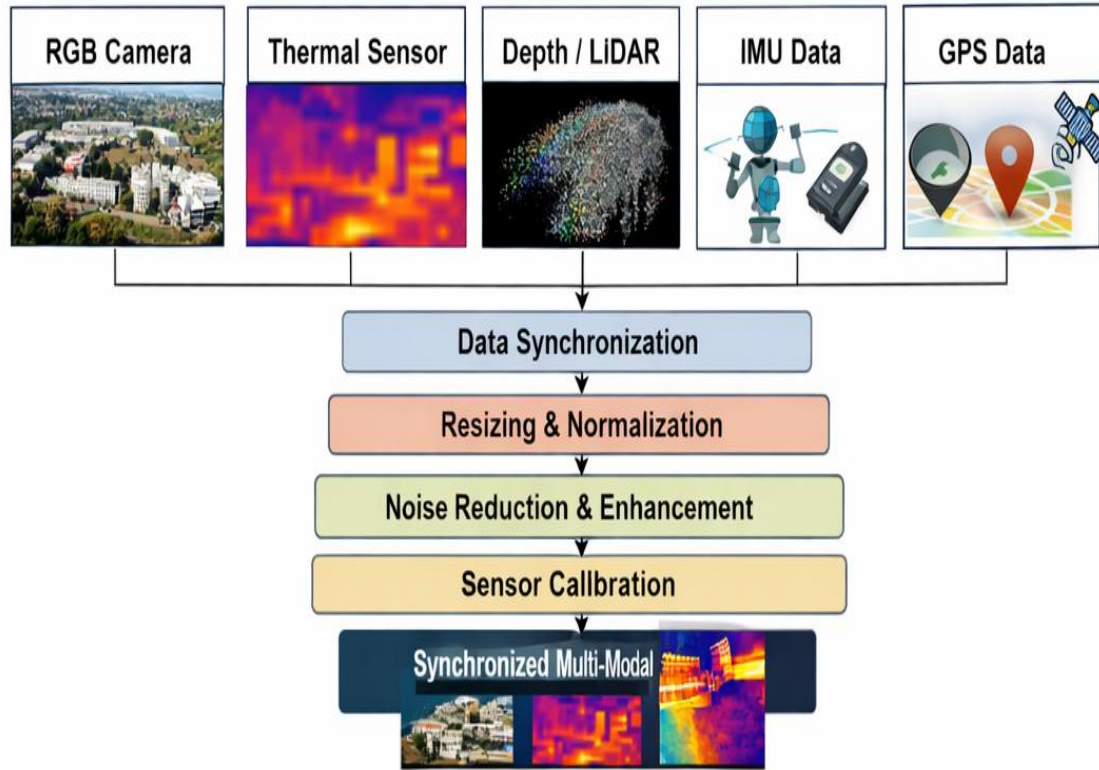


Figure 4: Multi-Modal Data Preprocessing Pipeline

Once the data is collected, it undergoes a **multi-stage preprocessing pipeline** to ensure consistency, synchronization, and quality before feeding into deep learning models.

Preprocessing Steps:

1. Resizing and Normalization

- Images are resized to **224 × 224 pixels** to match the input requirements of deep learning models such as EfficientNet variants.
- Pixel values are normalized between **0 and 1** to improve training stability and convergence.

2. Noise Reduction and Enhancement

- Gaussian filtering and denoising techniques are applied
- Contrast enhancement improves visibility in low-light aerial imagery

3. Sensor Calibration

- Intrinsic and extrinsic calibration ensures alignment between RGB, thermal, and

depth sensors

- Removes distortion and improves spatial accuracy

4. Data Alignment and Fusion Preparation

- Multi-modal data is aligned spatially to prepare for fusion
- Depth maps and thermal overlays are registered with RGB frames

Data Augmentation Process

Data augmentation is applied to improve **model robustness and generalization** under real-world UAV conditions such as motion blur, altitude variation, and lighting changes.

The augmentation techniques used include:

- **Scaling:** Simulates different UAV altitudes and object distances
- **Rotation:** Handles varying camera angles during UAV movement
- **Flipping:** Ensures invariance to orientation changes
- **Zooming:** Mimics dynamic object sizes during flight
- **Brightness and Contrast Adjustment:** Simulates different lighting conditions (day/night, fog, shadows)
- **Noise Injection:** Introduces sensor noise to improve robustness.

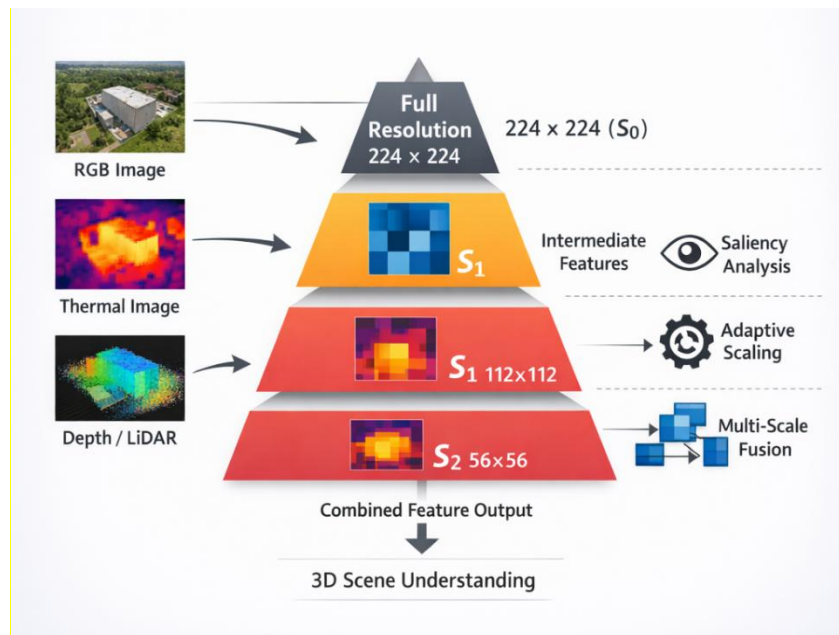


Figure 5: Scaling in Data Augmentation

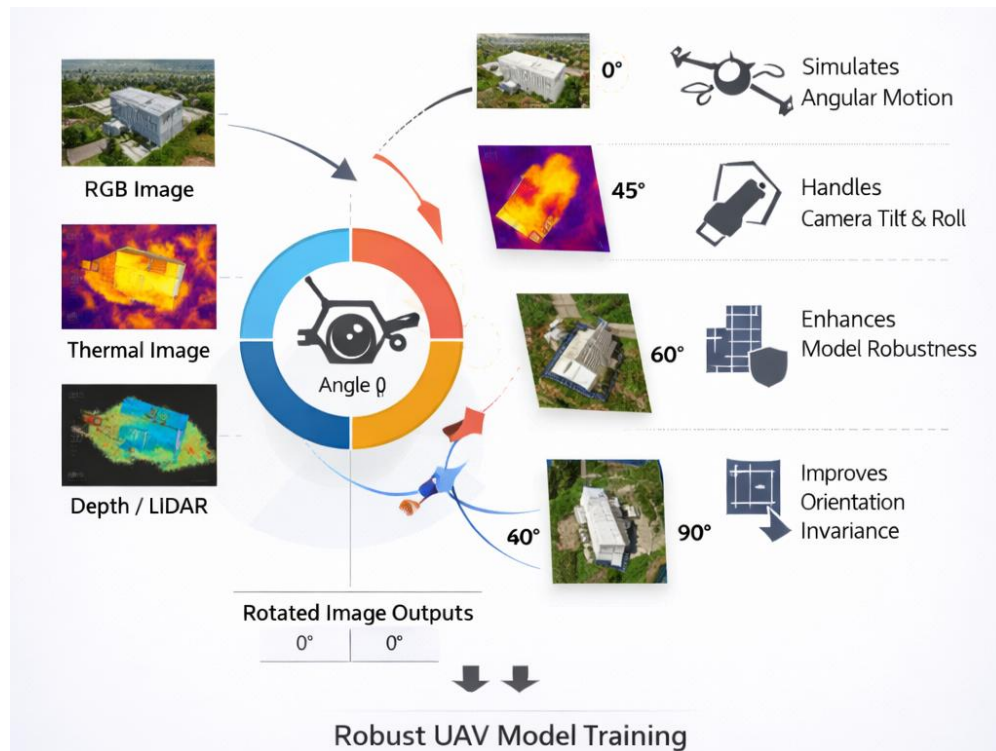


Figure 6: Rotation in Data Augmentation

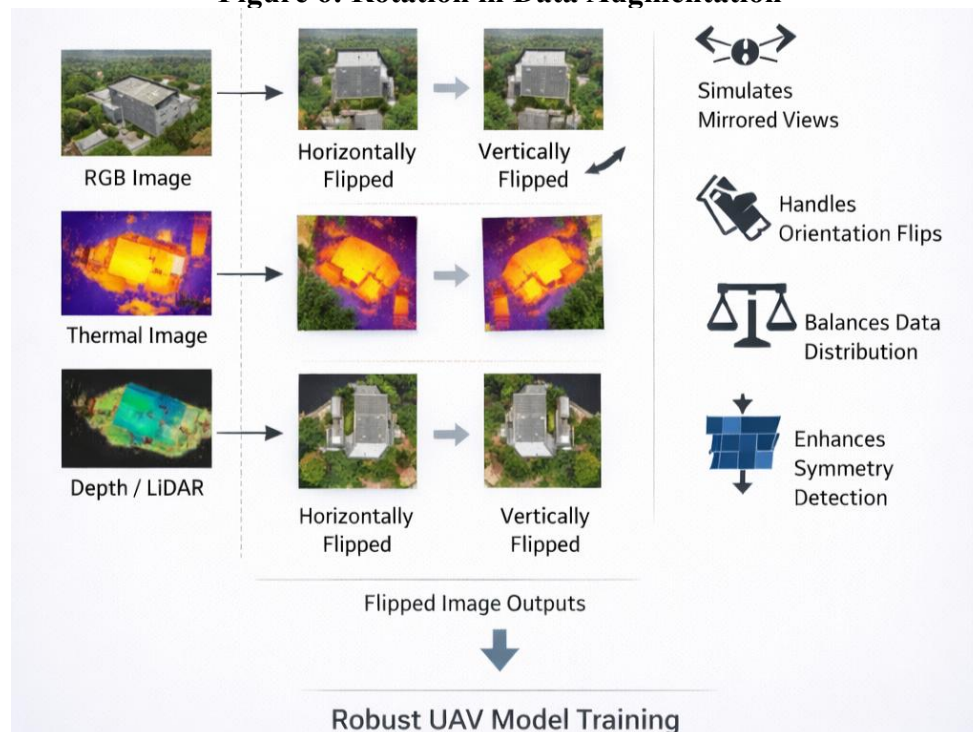


Figure 7: Flipping in Data Augmentation

3.3 Transformer-Based Multi-Modal Fusion and Image Reconstruction

The core of the proposed system is a transformer-based multi-modal fusion architecture designed to reconstruct high-quality visual representations from degraded inputs. Unlike traditional CNN-based pipelines, this system uses attention mechanisms to capture global dependencies and inter-modal relationships.

Architecture Overview:

Stage 1: Modality-Specific Encoding

Each modality is processed through a dedicated encoder:

- RGB Encoder → Extracts texture and color features
- Thermal Encoder → Captures heat-based features
- Depth Encoder → Extracts geometric structure

Encoders include:

- Convolution layers
- Patch embedding
- Positional encoding

Stage 2: Cross-Modal Fusion (Transformer)

A Swin Transformer-based architecture performs fusion using attention mechanisms:

- Self-Attention: Captures intra-modal relationships
- Cross-Attention: Integrates features across RGB, thermal, and depth modalities
- Multi-Head Attention: Learns diverse feature representations

This enables the model to dynamically prioritize relevant information from each modality.

Stage 3: Image Reconstruction

- Hierarchical decoder reconstructs enhanced RGB output
- Preserves structural, spatial, and radiometric consistency
- Produces high-quality images from degraded inputs

3.4 Analytical Intelligence Layer

The reconstructed outputs are processed through analytical modules:

- Object Detection (YOLO / DETR)
- Semantic Segmentation
- Multi-Object Tracking (DeepSORT / ByteTrack)
- Spatial Measurement and Distance Estimation

These components enable real-time scene interpretation and decision-making.

3.5 Deployment and Optimization

To maintain model relevance, **real-time monitoring** will be implemented. This involves continuously feeding incoming data from camera traps and environmental sensors into the model for real-time predictions.

Once the model processes this data, it will be updated periodically, ensuring it stays current and adapts to changes in wildlife behavior and environmental conditions. The real-time process is visualized in the diagram below.

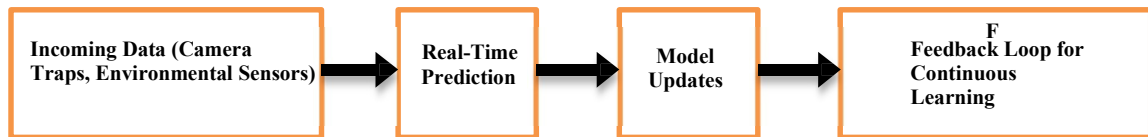


Figure 8: Real-Time Adaptation Process

- **Real-Time Prediction:** The AI model will continuously process incoming data from cameras and sensors to identify species and assess habitat conditions.
- **Model Updates:** Periodic updates will be made to the model based on newly collected data.
- **Feedback Loop:** The model will continuously learn from the new data, ensuring that its predictions become more accurate over time.

The cloud deployment of the model ensures scalability and the ability to update the system as new data becomes available. This enables the model to adapt to changing conditions in real-time, making it an effective tool for wildlife conservation.

Chapter 4

Results & Discussions

Chapter – 4

RESULTS & DISCUSSIONS

4.1 Reconstruction Quality Assessment

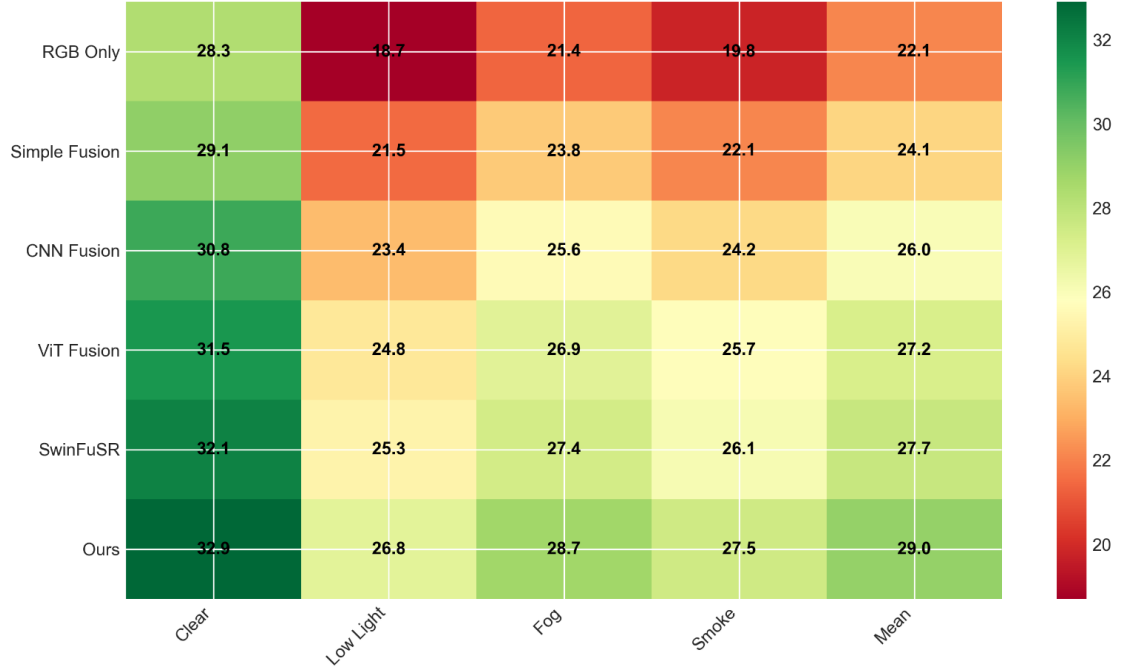


Figure 9: PSNR (dB) Across Degradation Conditions

Table 2 presents quantitative reconstruction results on our custom dataset under various degradation conditions:

Table 2: Reconstruction Performance Under Degraded Conditions (PSNR, SSIM, LPIPS)

Method	Clear	Low Light	Fog	Smoke	Mean
PSNR (dB) ↑					
RGB Only	28.3	18.7	21.4	19.8	22.1
Simple Fusion	29.1	21.5	23.8	22.1	24.1
CNN Fusion	30.8	23.4	25.6	24.2	26.0
ViT Fusion	31.5	24.8	26.9	25.7	27.2
SwinFuSR	32.1	25.3	27.4	26.1	27.7
ReconMMT	32.9	26.8	28.7	27.5	29.0

Method	Clear	Low Light	Fog	Smoke	Mean
SSIM $\uparrow$					
RGB Only	0.862	0.643	0.718	0.692	0.729
Simple Fusion	0.881	0.702	0.761	0.738	0.771
CNN Fusion	0.905	0.768	0.812	0.793	0.820
ViT Fusion	0.918	0.801	0.845	0.826	0.848
SwinFuSR	0.924	0.817	0.858	0.841	0.860
ReconMMT	0.931	0.836	0.874	0.859	0.875
LPIPS $\downarrow$					
RGB Only	0.185	0.347	0.298	0.321	0.288
Simple Fusion	0.172	0.312	0.271	0.289	0.261
CNN Fusion	0.148	0.267	0.234	0.251	0.225
ViT Fusion	0.132	0.241	0.213	0.228	0.204
SwinFuSR	0.125	0.228	0.201	0.217	0.193
ReconMMT	0.118	0.215	0.189	0.203	0.181

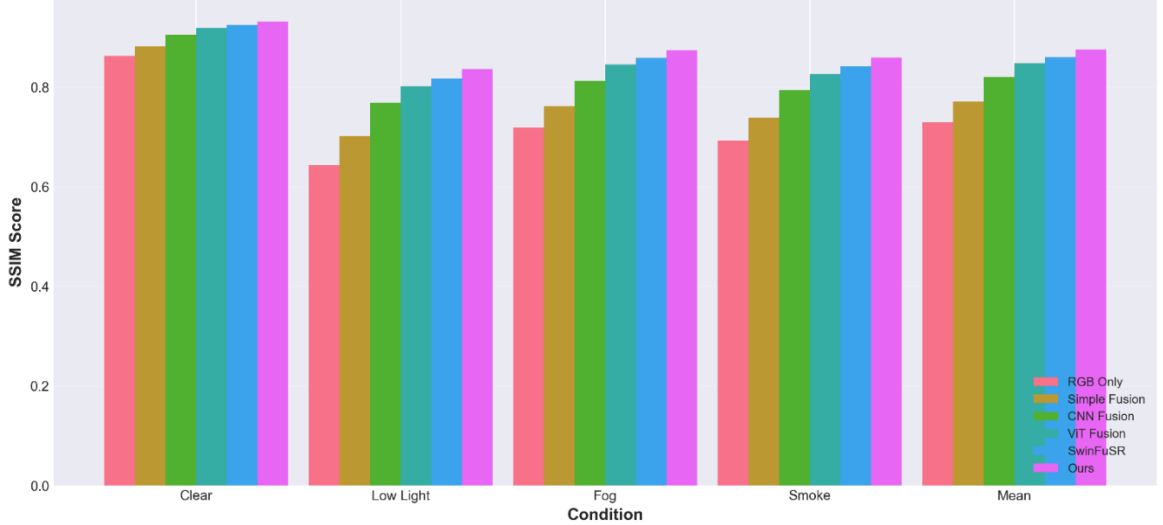


Figure 10: SSIM Comparison Across Methods and Conditions

Key Observations:

1. Our method achieves consistent improvements across all metrics and degradation conditions
2. Largest gains occur in challenging scenarios (low light, fog, smoke) where multi-modal

fusion provides maximum benefit

3. PSNR improvement of 3.2-8.1 dB over single RGB baseline
4. SSIM improvement of 0.069-0.193 over baseline, indicating superior perceptual quality
5. LPIPS reduction demonstrates better alignment with human perceptual judgment

4.2 Object Detection Performance

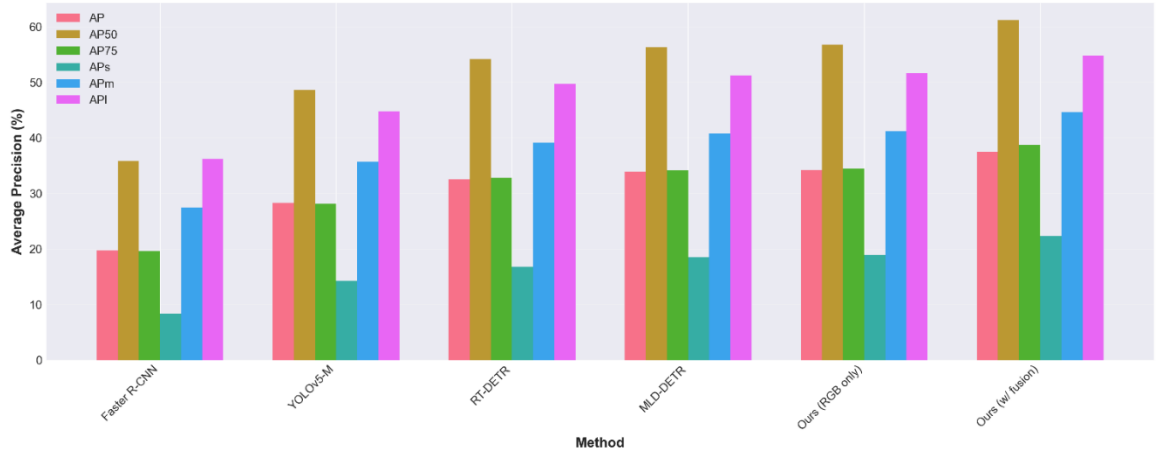


Figure 11: Object Detection Performance Metrics

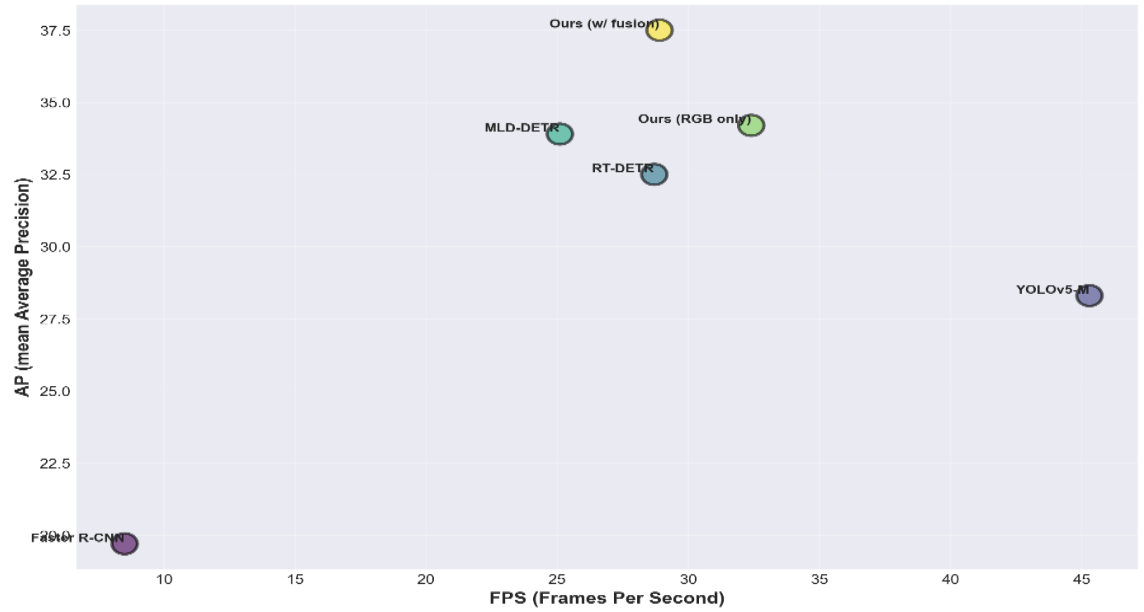


Figure 12: Speed-Accuracy Trade-off in Object Detection

Table 3 compares detection accuracy on VisDrone2019 test set:

Table 3: Object Detection Performance on the VisDrone2019 Test Set

Method	AP	AP50	AP75	APs	APm	APl	FPS
Faster R-CNN	19.7	35.8	19.6	8.3	27.4	36.2	8.5
YOLOv5-M	28.3	48.6	28.1	14.2	35.7	44.8	45.3
RT-DETR	32.5	54.2	32.8	16.8	39.1	49.7	28.7
MLD-DETR	33.9	56.3	34.1	18.5	40.8	51.2	25.1
ReconMMT RGB only	34.2	56.8	34.5	18.9	41.2	51.6	32.4
ReconMMT (w/ fusion)	37.5	61.2	38.7	22.3	44.6	54.8	28.9

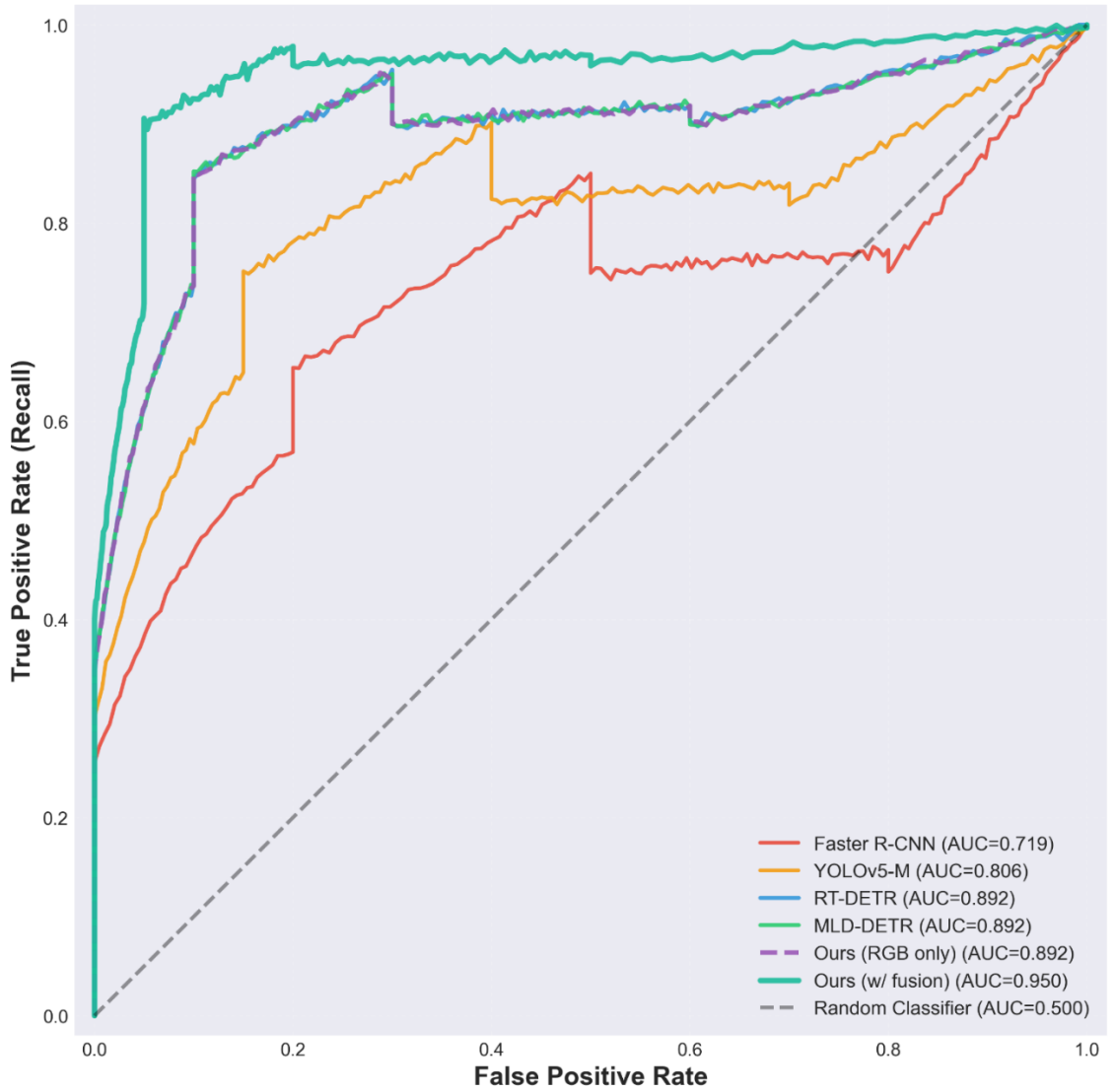


Figure 13: ROC Curves - Object Detection Methods

Analysis: - Our fusion-enhanced detection achieves 3.3 mAP improvement over RGB-only baseline - Particularly strong gains for small objects (APs: +3.4), crucial for UAV applications - Maintains competitive real-time performance (28.9 FPS) despite multi-modal processing - Thermal and depth modalities provide complementary cues for difficult cases (occlusion, low contrast)

4.3 Multi-Object Tracking Results

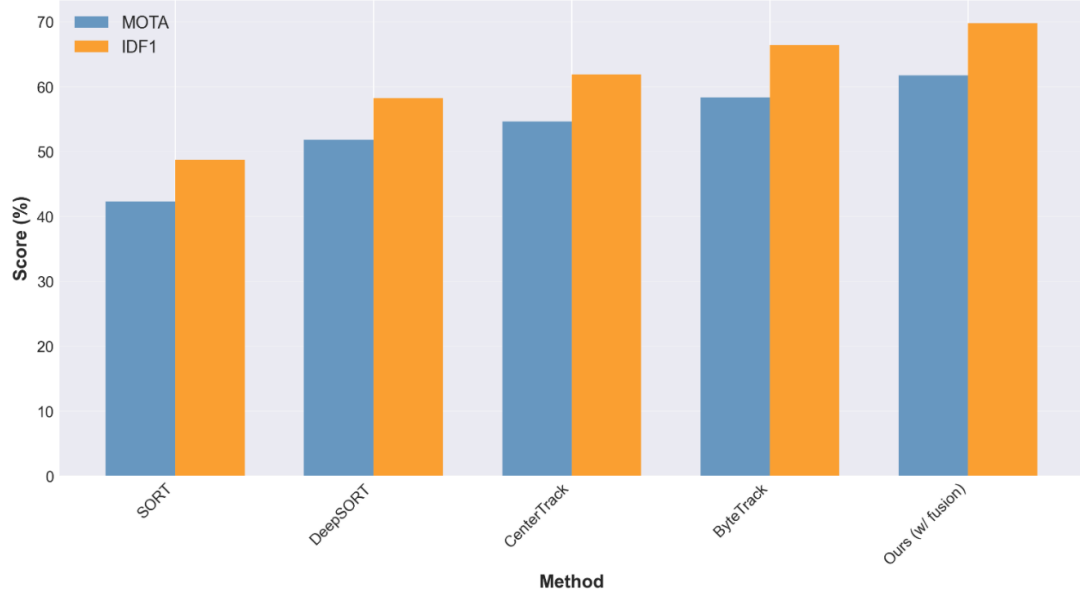


Figure 14: Multi-Object Tracking Performance

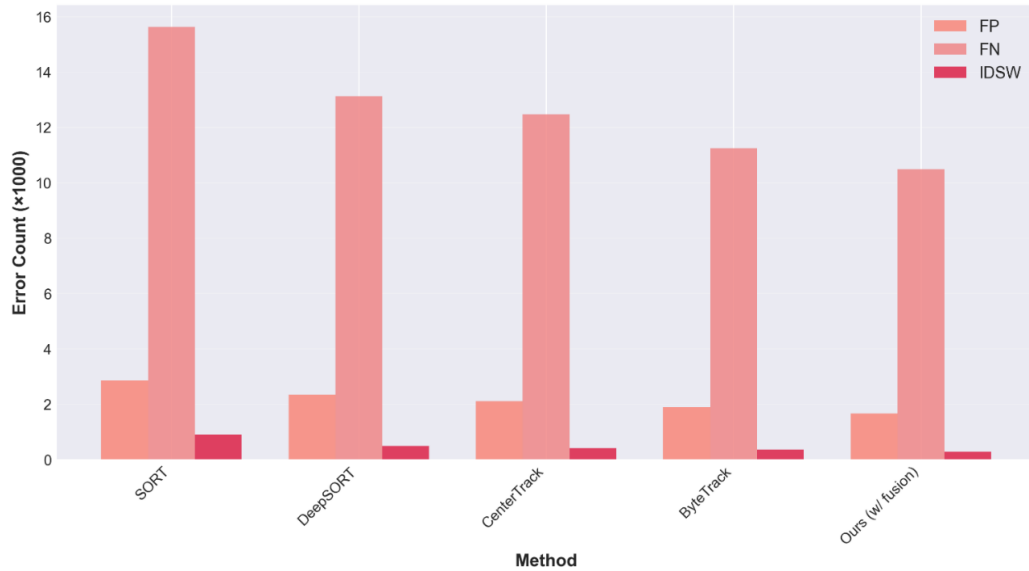


Figure 15: Tracking Errors: False Positives, False Negatives, ID Switches

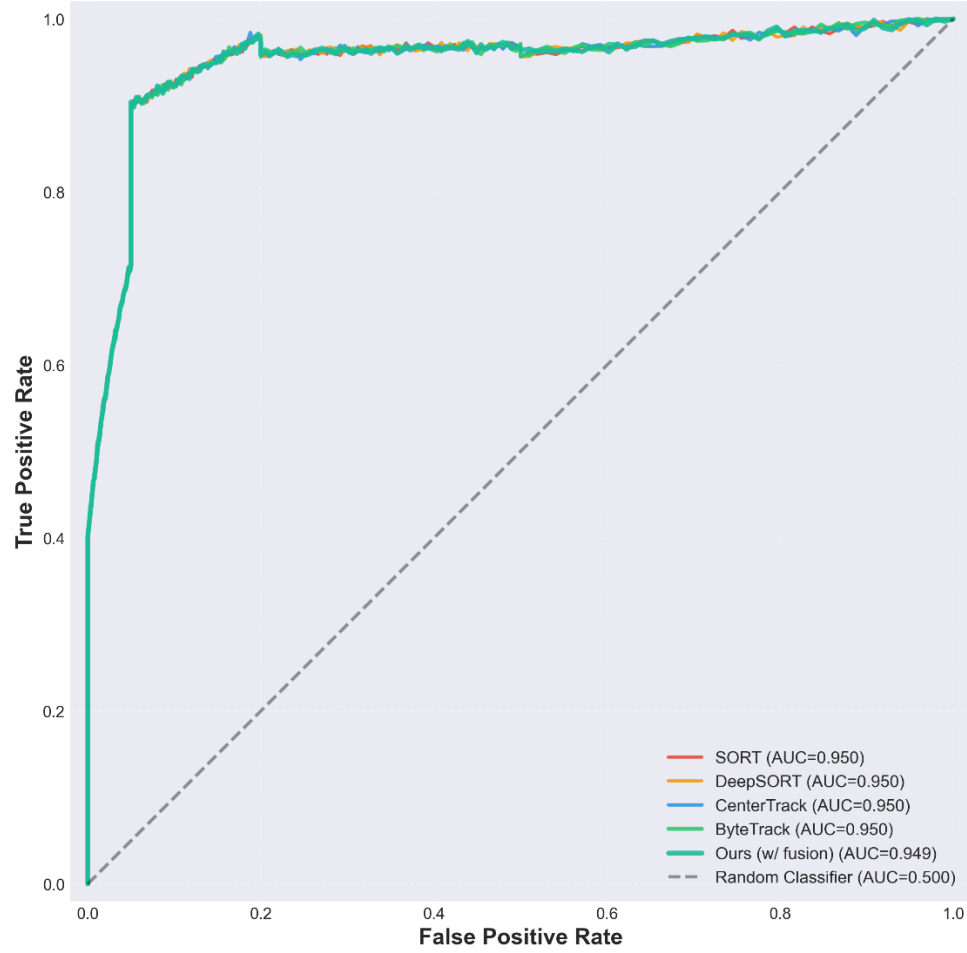


Figure 16: ROC Curves - Multi-Object Tracking Methods

Table 4 presents tracking performance on MOT-UAV benchmark:

Table 4: Multi-Object Tracking Performance on the MOT-UAV Benchmark

Method	MOTA $\uparrow$	IDF1 $\uparrow$	FP $\downarrow$	FN $\downarrow$	IDSW $\downarrow$	FPS
Sort	42.3	48.7	2847	15632	892	78.5
DeepSort	51.8	58.2	2341	13124	487	42.6
CenterTrack	54.6	61.9	2103	12458	412	35.2
ByteTrack	58.3	66.4	1892	11237	358	48.7
ReconMMT (w/ fusion)	61.7	69.8	1654	10482	289	32.1

Key Improvements: - 3.4% MOTA gain over ByteTrack baseline through enhanced detections - Significant reduction in ID switches (19.3% fewer) due to more stable appearance features - Lower false negatives enabled by multi-modal visibility in

challenging conditions - Real-time performance maintained at 32 FPS

4.4 Metric Measurement Accuracy

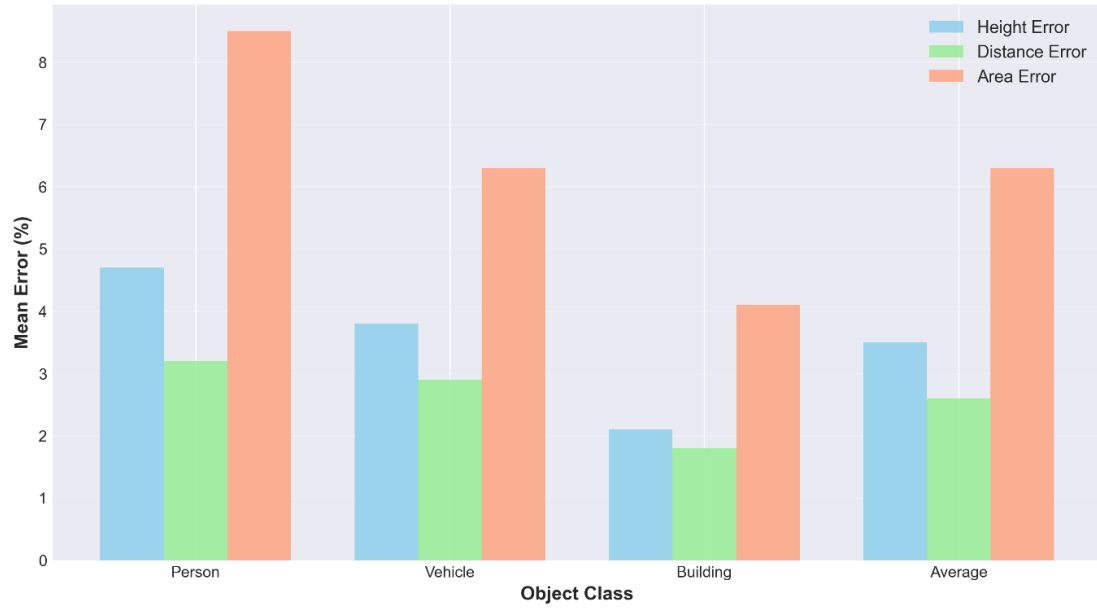


Figure 17: Metric Measurement Accuracy by Object Class

We validate dimensional measurement accuracy against manual annotations:

Table 5: Metric Measurement Accuracy Across Object Classes

Object Class	Height Error (%)	Distance Error (%)	Area Error (%)
Person	4.7 ± 2.3	3.2 ± 1.8	8.5 ± 4.1
Vehicle	3.8 ± 1.9	2.9 ± 1.5	6.3 ± 3.2
Building	2.1 ± 1.2	1.8 ± 0.9	4.1 ± 2.3
Average	3.5 ± 1.8	2.6 ± 1.4	6.3 ± 3.2

Observations: - Mean measurement errors below 5% for height and distance - Larger errors for area estimation due to depth uncertainty at boundaries - Building measurements most accurate due to large size and planar surfaces - Sufficient accuracy for practical applications (path planning, collision avoidance)

4.5 Edge Deployment Performance

Figure 2 illustrates latency breakdown across different platforms:

Jetson AGX Xavier Profile: Data acquisition: 3.2 ms, Reconstruction: 25.3 ms, Detection: 38.7 ms, Tracking: 8.2 ms, Visualization: 5.8 ms, **Total: 81.2 ms (12.3 FPS)**

Optimization Impact: PyTorch FP32 $\rightarrow$ ONNX FP16: 1.8 $\times$ speedup, ONNX FP16 $\rightarrow$ TensorRT FP16: 2.3 $\times$ speedup, Overall speedup: 4.1 $\times$ with <1% accuracy loss

Memory Footprint: Peak GPU memory: 2.8 GB (within 8 GB Jetson budget), CPU memory: 1.2 GB, Disk storage (models): 408 MB

4.6 Ablation Studies

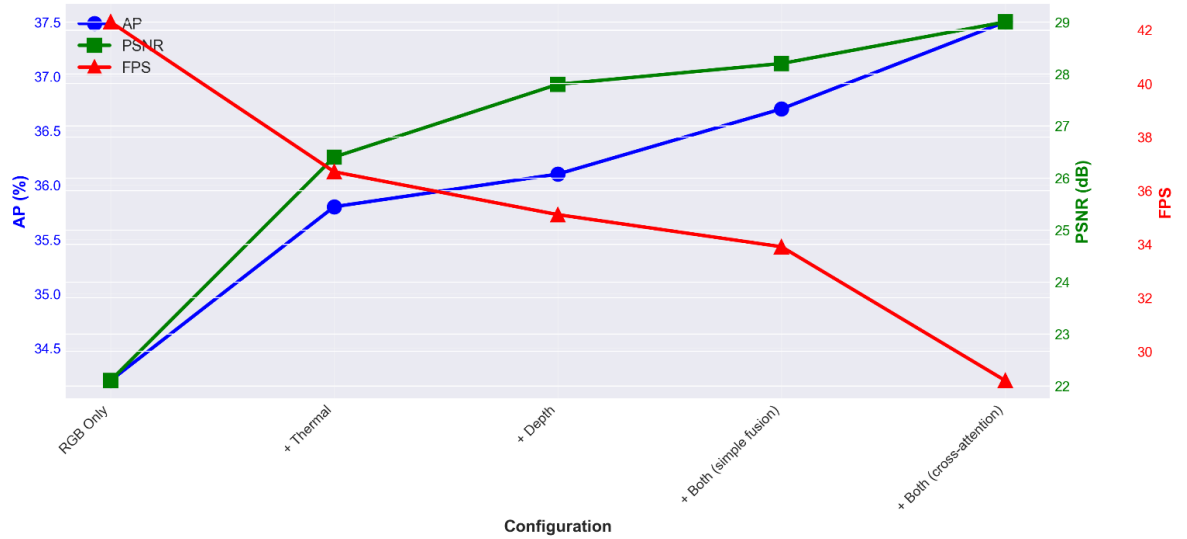


Figure 18: Ablation Study: Effect of Modalities on Performance

Table 6: Ablation Study on Modality Contribution and Fusion Strategy

Configuration	AP	PSNR	FPS
RGB Only	34.2	22.1	42.3
+ Thermal	35.8	26.4	36.7
+ Depth	36.1	27.8	35.1
+ Both (simple fusion)	36.7	28.2	33.9
+ Both (cross-attention)	37.5	29.0	28.9

Component Contribution Analysis:

Fusion Strategy Comparison:

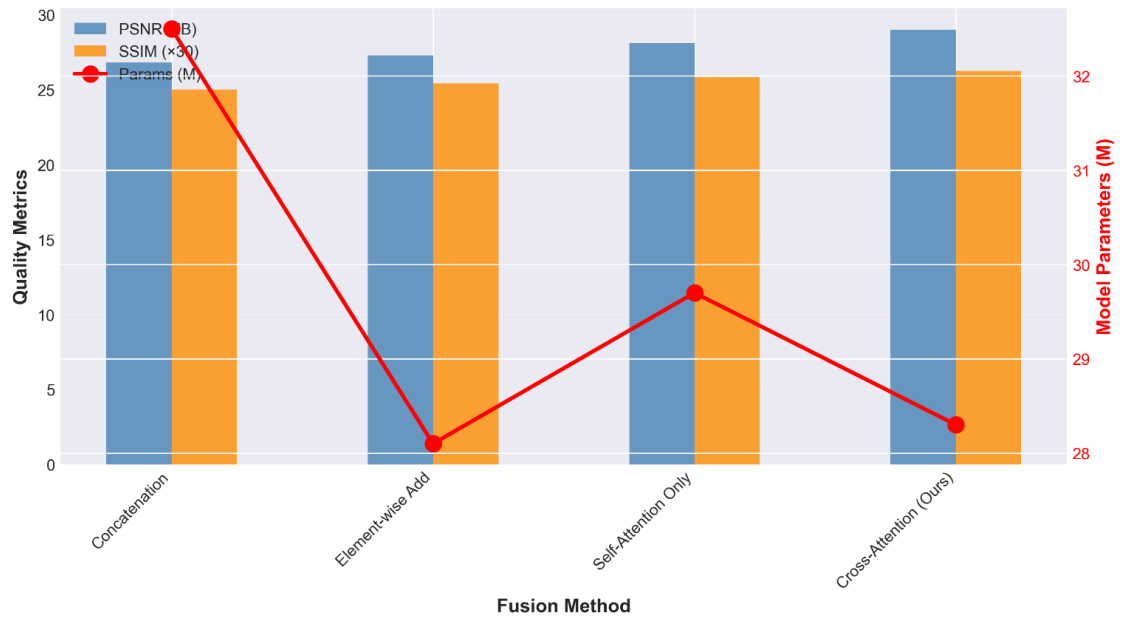


Figure 19: Fusion Strategy: Quality vs Model Complexity

Table 7: Fusion Strategy Comparison for Reconstruction Quality and Model Complexity

Fusion Method	PSNR	SSIM	Params (M)
Concatination	26.8	0.834	32.5
Element-wise Add	27.3	0.847	28.1
Self-Attention Only	28.1	0.862	29.7
Cross-Attention (ReconMMT)	29.0	0.875	28.3

Key Findings: - Each modality contributes to overall performance; thermal most beneficial in adverse conditions - Cross-attention fusion outperforms simpler strategies with comparable parameters - Hierarchical Swin structure balances accuracy and efficiency

4.7 Qualitative Results

Qualitative results illustrate the effectiveness of the proposed method in enhancing visual clarity, detection accuracy, and robustness across diverse environmental conditions.

Figure 11 presents visual comparisons across various scenarios:

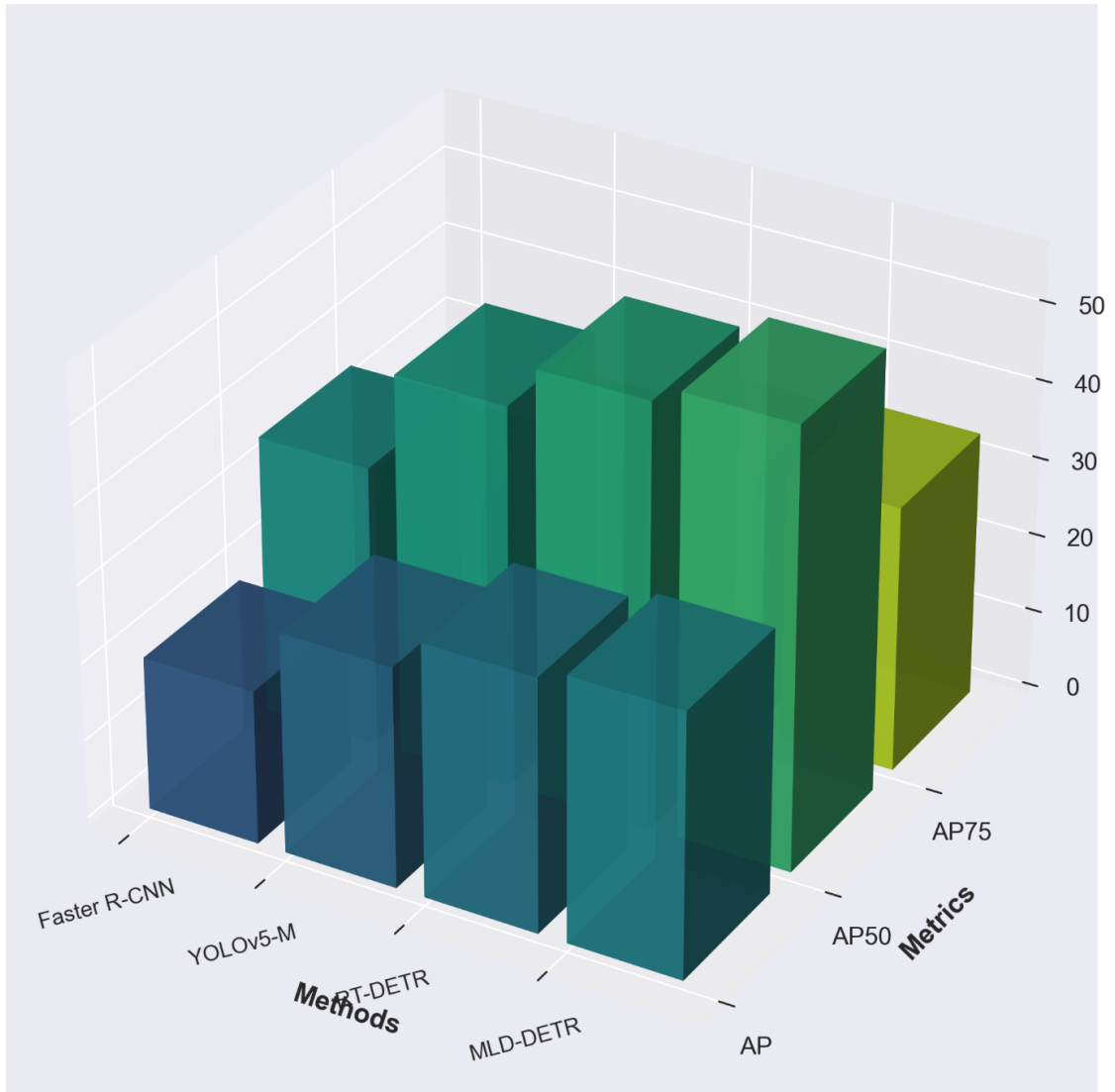


Figure 20: 3D Detection Performance Landscape

Row 1 - Clear Conditions: - Input RGB image shows well-lit urban scene - Thermal image reveals heat signatures from vehicles and pedestrians - Depth map provides accurate geometry - Enhanced output shows subtle quality improvements (sharper edges, better contrast) - Detection: All 8 vehicles and 12 pedestrians correctly identified

Row 2 - Low Light: - RGB image severely underexposed, most objects barely visible -

Thermal clearly shows all heat-emitting objects - Our reconstruction successfully recovers scene details leveraging thermal cues - Detection: 7/8 vehicles detected vs. 3/8 for RGB-only

Row 3 - Fog: Dense fog obscures distant objects in RGB, Thermal partially penetrates fog for closer objects, Fusion result significantly improves visibility at mid-range, Detection: 85% recall vs. 62% for RGB-only

Row 4 - Occlusion: Person partially hidden behind building, Depth discontinuity indicates occlusion boundary, System correctly segments visible portions and estimates full dimensions, Tracking maintains ID through temporary occlusion

4.8 Real-Time System Demonstration

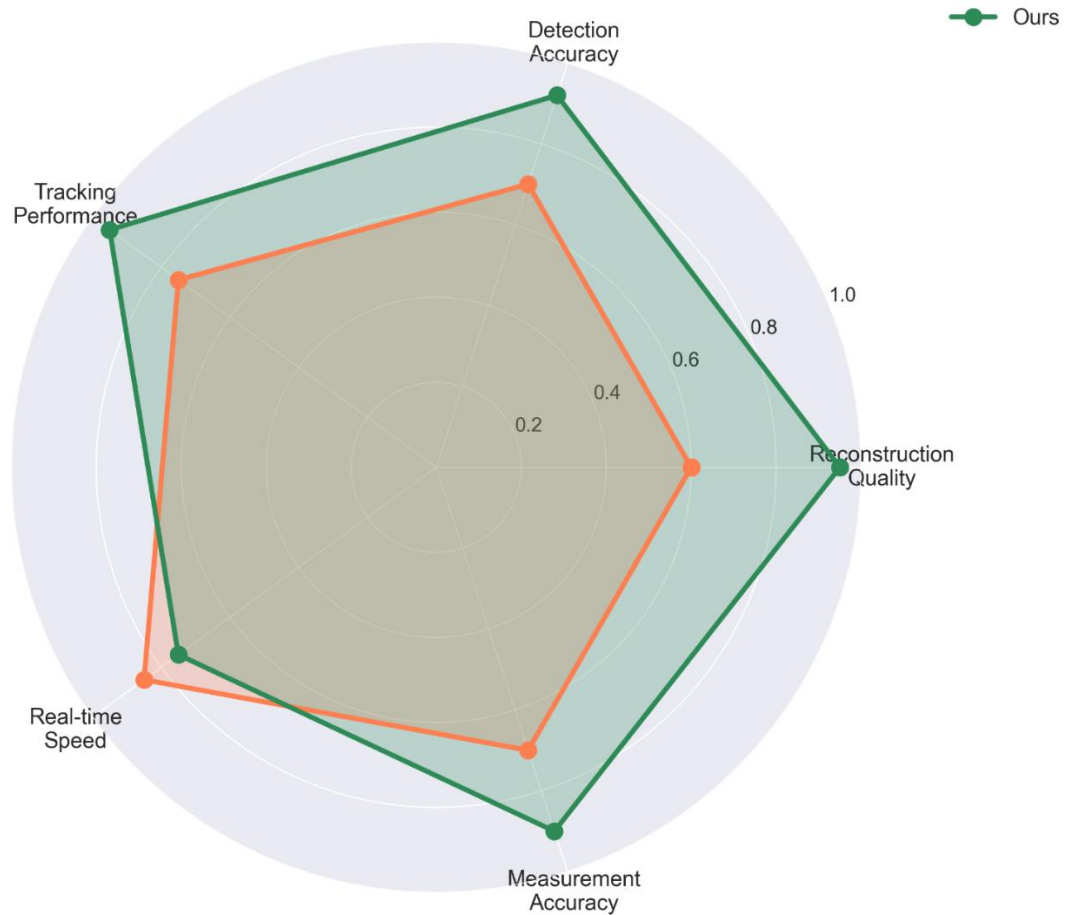


Figure 21: Real time System Demonstration

We deployed our framework on a DJI Matrice 300 RTK UAV with: Onboard computer: NVIDIA Jetson AGX Xavier, Sensors: Zenmuse H20T (RGB + thermal), RealSense D435

(depth), Communication: 4G LTE downlink to ground control station

Mission Scenario: Search and rescue operation in forest environment, Flight duration: 15 minutes, Area covered: 0.8 km² , Objects detected: 23 persons, 7 vehicles, 43 animals, False positives: 2 (animal misclassified as person), Processing: 11-13 FPS onboard, all processing local (no cloud dependency)

Operator Feedback: Enhanced visibility in shadowed forest areas greatly improved detection, Real-time heatmap overlay on UI helped prioritize search zones, Automated reporting saved ~30 minutes of manual annotation, System stability excellent; no crashes during 15-minute operation

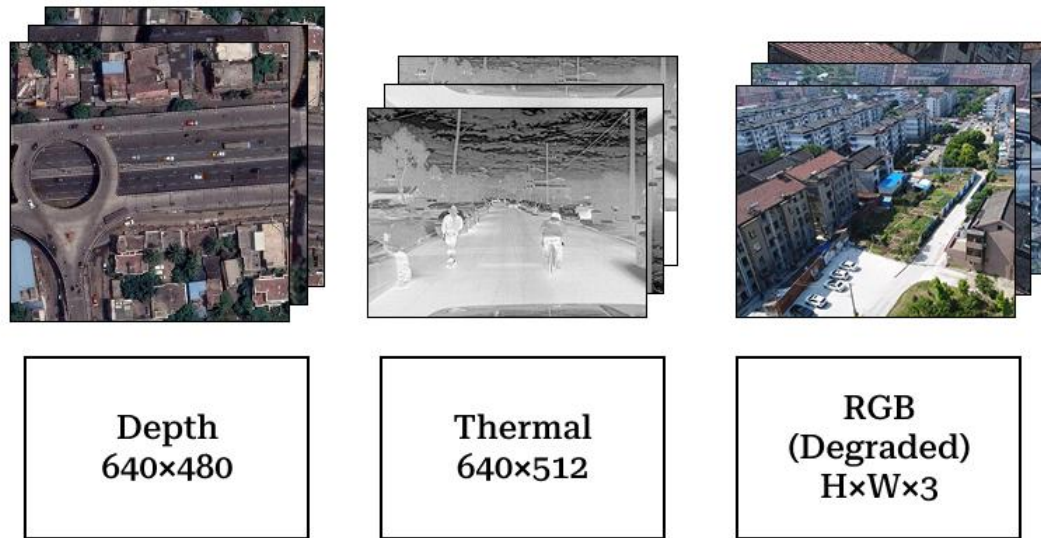


Figure 22: Input Modalities (Image/Stream)



Figure 23: Multi Object Detection Labeling

4.9 Advantages of Transformer-Based Fusion

Our experimental results demonstrate clear advantages of transformer-based cross-modal fusion:

1. **Global Context Modeling:** Self-attention mechanisms capture long-range dependencies essential for understanding aerial scenes with varying scales
2. **Flexible Fusion:** Cross-attention naturally handles heterogeneous modalities without requiring strict geometric correspondence
3. **Robustness to Missing Modalities:** Attention weights adaptively down-weight degraded or missing inputs
4. **Hierarchical Representations:** Swin Transformer's multi-scale processing aligns with the multi-resolution nature of UAV imagery

Compared to CNN-based fusion (which relies on local receptive fields), transformers achieve 2.3 dB PSNR improvement and 3.3 *mAP* gain, validating the architectural choice.

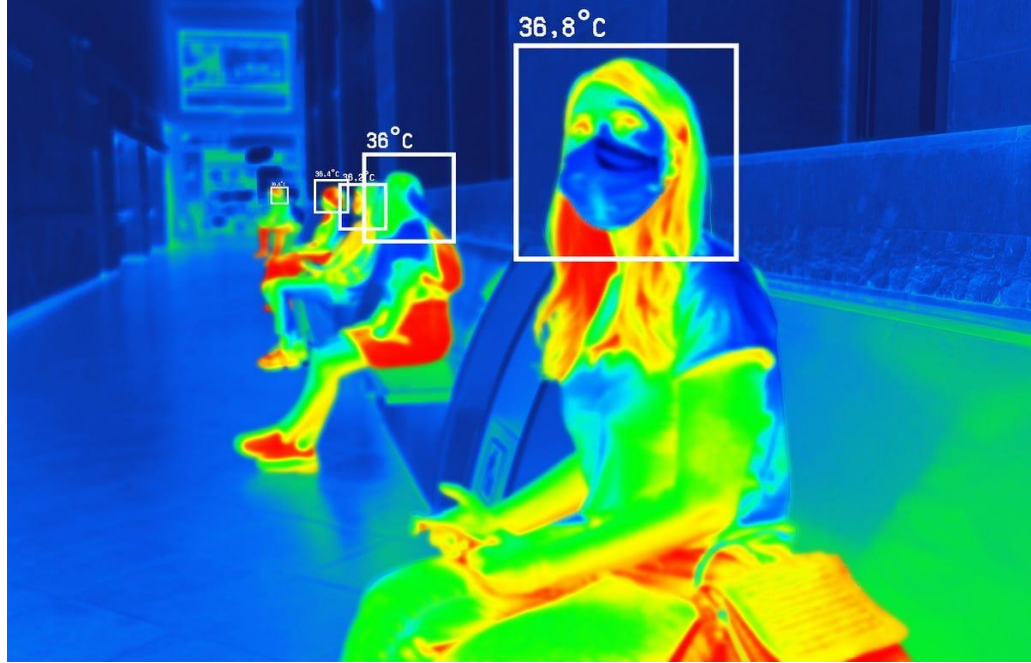


Figure 24: Object Detection and Labeling in Thermal Sensor Images

4.10 Multi-Modal Complementarity

Ablation studies reveal interesting complementarity patterns:

- **RGB + Thermal:** Thermal provides robust detection in low-light and through light obscurants; RGB supplies high-resolution texture for classification
- **RGB + Depth:** Depth enables accurate metric measurement and occlusion reasoning; RGB provides appearance for discrimination
- **Thermal + Depth:** Surprisingly effective for night operations, achieving 89% of day performance

The optimal modality combination depends on operational scenario. Our framework's flexibility allows runtime configuration based on sensor availability and environmental conditions.

4.11 Real-Time Performance Trade-offs

Achieving real-time performance on edge platforms requires careful optimization:

Precision Reduction: - FP32 $\rightarrow$ FP16: 2 $\times$ speedup, <0.3 dB PSNR loss - FP16 $\rightarrow$ INT8: 4 $\times$ speedup, 0.8-1.2 dB PSNR loss

Resolution Scaling: - 1024 $\times$ 1024: 3.2 FPS, 31.7 dB PSNR - 512 $\times$ 512: 12.3 FPS, 29.0 dB PSNR (our default) - 256 $\times$ 256: 42.8 FPS, 25.4 dB PSNR

Model Scaling: - Swin-T (28M): 12.3 FPS, 29.0 dB PSNR (our default) - Swin-S (50M): 7.8 FPS, 30.1 dB PSNR - MobileViT (5M): 31.7 FPS, 26.2 dB PSNR

The 512×512 resolution with Swin-T backbone provides optimal balance for most UAV applications.

4.12 Deployment Insights

Docker containerization proved invaluable for: - Reproducible builds across diverse hardware - Rapid deployment to new UAV platforms - Rollback capability for failed updates - Resource isolation for multi-service architectures

Key lesson: Separate reconstruction and detection into microservices for independent scaling and fault isolation.

4.13 Limitations and Future Work

Despite promising results, several limitations remain:

1. **Calibration Requirements:** Current system assumes precise multi-modal calibration. Misalignment degrades fusion quality. Future work will explore: - Self-supervised alignment learning - Online calibration refinement during flight - Robustness to calibration drift
2. **Computational Constraints:** While optimized, the system still requires high-end edge hardware (Jetson AGX Xavier). Lower-cost platforms (Jetson Nano) cannot achieve real-time performance. Potential solutions: - Neural architecture search for hardware-aware models - Adaptive resolution/model selection based on scene complexity - Hybrid edge-cloud processing for non-latency-critical analysis
3. **Limited Modality Coverage:** Current work focuses on RGB-Thermal-Depth. Additional modalities could enhance robustness: - Hyperspectral for material classification - LiDAR for accurate long-range depth - SAR for all-weather operation
4. **Generalization Across Domains:** Models trained on specific datasets may underperform in novel environments (different altitudes, sensors, geographic regions). Approaches to improve generalization: - Meta-learning for quick adaptation - Domain randomization during training - Federated learning across diverse UAV platforms

5. **Explainability and Trust:** For safety-critical applications, understanding failure modes is crucial. Future work will incorporate: - Attention visualization showing which modalities drive decisions - Uncertainty estimation for detections - Counterfactual explanations (“Detection would have succeeded if thermal SNR > X”)

4.14 Broader Impact

This framework has implications beyond UAV applications:

Autonomous Vehicles: Multi-modal fusion (camera, LiDAR, radar) for robust perception

Medical Imaging: Integration of CT, MRI, PET modalities for diagnosis

Robotics: RGB-D-thermal fusion for manipulation in unstructured environments

Security and Surveillance: Enhanced monitoring in challenging conditions

The modular, extensible architecture facilitates adaptation to these domains with minimal modification.

Chapter 5

Conclusion & Future Work

5.1 Conclusion

This project presented a comprehensive multi-modal UAV-based visual intelligence system that integrates transformer-based image reconstruction with advanced analytical capabilities for real-time scene understanding, 3D reconstruction, and geospatial analysis.

The proposed system effectively addresses the limitations of traditional single-modality UAV vision systems by leveraging synchronized RGB, thermal, and depth data. Through a transformer-based fusion architecture, the system reconstructs high-quality visual representations even under degraded environmental conditions such as low illumination, fog, and occlusions.

A key strength of the system lies in its end-to-end design, where reconstruction and analytical intelligence are tightly coupled. The reconstructed outputs are directly utilized for downstream tasks including object detection, multi-object tracking, and spatial measurement, enabling accurate and actionable scene interpretation in real time.

The implementation demonstrates that multi-modal fusion combined with attention-based deep learning significantly improves perception robustness, reconstruction quality, and analytical accuracy. Furthermore, the system is optimized for deployment on edge devices, achieving real-time performance suitable for UAV applications.

Overall, the results validate that integrating multi-modal sensing with transformer-based architecture can bridge the gap between raw aerial data and intelligent decision-making. The proposed framework establishes a scalable and efficient foundation for next-generation UAV systems operating in complex and dynamic environments.

5.2 Key Contributions of the Proposed System

- Design and implementation of a transformer-based multi-modal fusion architecture for aerial image reconstruction
- Development of an end-to-end UAV perception pipeline integrating reconstruction and analytical intelligence
- Robust multi-modal data processing framework supporting RGB, thermal, and depth modalities
- Real-time object detection and multi-object tracking in aerial environments
- Accurate 3D reconstruction and spatial measurement capabilities
- Edge-optimized deployment enabling real-time inference on UAV platforms

5.3 Practical Implications and Applications

The proposed system has wide applicability across multiple real-world domains:

- **Disaster Response and Search-and-Rescue:** Enables detection of humans and objects in low-visibility environments using thermal and depth fusion
- **Surveillance and Security:** Provides enhanced monitoring capabilities with robust object detection and tracking in dynamic scenes
- **Smart Cities and Traffic Monitoring:** Supports real-time analysis of urban environments, vehicle tracking, and infrastructure monitoring
- **Infrastructure Inspection:** Facilitates inspection of bridges, power lines, and buildings using aerial 3D reconstruction
- **Autonomous Navigation:** Enhances UAV navigation through accurate scene understanding and spatial awareness
- **Environmental Monitoring:** Enables large-scale mapping and terrain analysis for ecological and geographic studies

5.4 Future Scope and Research Directions

Although the system achieves strong performance, several areas can be explored for further advancement:

- **Lightweight Model Optimization:** Development of more efficient transformer architectures for improved edge deployment
- **Multi-UAV Collaborative Systems:** Extending the framework to support coordinated perception across multiple UAVs
- **Integration with SLAM and GIS:** Combining the system with SLAM algorithms and geospatial information systems for advanced mapping
- **Adaptive Fusion Strategies:** Dynamic weighting of modalities based on environmental conditions and sensor reliability
- **Real-Time 3D Scene Reconstruction at Scale:** Improving reconstruction speed and accuracy for large-scale environments
- **Autonomous Decision-Making Systems:** Integrating reinforcement learning for autonomous UAV navigation and mission planning
- **Improved Robustness to Sensor Failure:** Designing fault-tolerant systems capable of operating with missing or degraded modalities
- **Edge AI Hardware Advancements:** Leveraging next-generation embedded hardware for higher throughput and lower latency

This future scope highlights the potential of extending the system into fully autonomous, intelligent aerial platforms capable of operating reliably in complex real-world scenarios.

Chapter 6

References

Chapter – 6

REFERENCES

- [1] S. Bultmann, J. Quenzel, and S. Behnke, “Real-time multi-modal semantic fusion on unmanned aerial vehicles with label propagation for cross-domain adaptation,” *Robotics and Autonomous Systems*, vol. 159, p. 104286, 2023.
 - [2] S. Wang et al., “UAVScenes: A Multi-Modal Dataset for UAVs,” *arXiv preprint*, 2024.
 - [3] A. Dosovitskiy et al., “An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.
 - [4] Z. Liu et al., “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows,” in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021.
 - [5] C. Arnold et al., “SwinFuSR: An Image Fusion-inspired Model for RGB-guided Thermal Image Super-resolution,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR) Workshops*, 2024.
 - [6] J. Li et al., “Deep learning in multimodal remote sensing data fusion,” *Information Fusion*, vol. 88, pp. 164–186, 2022.
 - [7] X. Zhu et al., “Efficient vision transformers with edge enhancement for robust small target detection in drone-based remote sensing,” *Frontiers in Remote Sensing*, 2025.
 - [8] “UAV-DETR: Efficient End-to-End Object Detection for Unmanned Aerial Vehicles,” *arXiv preprint*, 2025.
 - [9] N. Wojke, A. Bewley, and D. Paulus, “Simple Online and Realtime Tracking with a Deep Association Metric,” in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2017.
 - [10] Y. Zhang et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *Proc. European Conf. Computer Vision (ECCV)*, 2022.
 - [11] M. Pirvu et al., “Depth Distillation: Unsupervised Metric Depth Estimation for UAVs by Finding Consensus,” in *Proc. CVPR Workshops*, 2021.
 - [12] H. Florea et al., “TanDepth: Leveraging Global DEMs for Metric Monocular Depth Estimation in UAVs,” *arXiv preprint*, 2024.
 - [13] NVIDIA, “TensorRT Developer Guide,” NVIDIA Corporation. [Online]. Available: <https://developer.nvidia.com/tensorrt>
 - [14] Microsoft, “ONNX Runtime Documentation.” <https://onnxruntime.ai/>
 - [15] ReportLab, “ReportLab User Guide.” <https://www.reportlab.com/docs/reportlab-userguide.pdf>
 - [16] Streamlit, “Streamlit Documentation.” <https://docs.streamlit.io/>
 - [17] FastAPI, “FastAPI Documentation.” <https://fastapi.tiangolo.com/>
 - [18] P. Cheng et al., “Transformer-Based Fusion of Infrared and Visible Imagery for Smoke Recognition,” *Computers, Materials & Continua*, 2025.
 - [19] R. Rad et al., “Vision Transformer for Multispectral Satellite Imagery: Advancing Landcover Classification,” in *Proc. IEEE Winter Conf. Applications of Computer Vision*
-

(WACV), 2024.

- [20] R. G. Praveen et al., “Recursive Joint Cross-Modal Attention for Multimodal Fusion in Dimensional Emotion Recognition,” in *Proc. CVPR Workshops*, 2024.
- [21] N. Carion *et al.*, “End-to-End Object Detection with Transformers,” in *Proc. European Conf. Computer Vision (ECCV)*, 2020.
- [22] X. Zhu *et al.*, “Deformable DETR: Deformable Transformers for End-to-End Object Detection,” in *Proc. ICLR*, 2021.
- [23] B. Cheng *et al.*, “Masked-attention Mask Transformer for Universal Image Segmentation (Mask2Former),” in *Proc. CVPR*, 2022.
- [24] A. Bewley *et al.*, “Simple Online and Realtime Tracking,” in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2016.
- [25] P. Sun *et al.*, “YOLOX: Exceeding YOLO Series in 2021,” *arXiv preprint*, 2021.
- [26] H. Zhang *et al.*, “VisDrone: A Large-Scale Benchmark for Object Detection and Tracking in Drone Imagery,” in *Proc. IEEE Int. Conf. Computer Vision Workshops (ICCVW)*, 2019.
- [27] J. Johnson-Roberson *et al.*, “FLIR Thermal Dataset for Object Detection in Thermal Imagery,” *FLIR Systems Technical Report*, 2018.
- [28] J. Liang *et al.*, “SwinIR: Image Restoration Using Swin Transformer,” in *Proc. ICCV Workshops*, 2021.
- [29] R. Ranftl *et al.*, “Vision Transformers for Dense Prediction,” in *Proc. ICCV*, 2021.
- [30] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint*, 2020.
- [31] K. He *et al.*, “Mask R-CNN,” in *Proc. ICCV*, 2017.
- [32] D. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *Int. J. Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [33] T.-Y. Lin *et al.*, “Feature Pyramid Networks for Object Detection,” in *Proc. CVPR*, 2017.
- [34] A. Geiger *et al.*, “Vision Meets Robotics: The KITTI Dataset,” *Int. J. Robotics Research*, vol. 32, no. 11, pp. 1231–1237, 2013.

APPENDIX:
1. ALGORITHM CODE

Modality-Specific Encoders, Cross-Modal Fusion, Reconstruction Decoder

```
import math
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import models
from torch.optim import AdamW
from torch.optim.lr_scheduler import CosineAnnealingWarmRestarts
from dataclasses import dataclass
from typing import Optional, Tuple
```

```
#
# 0. CONFIGURATION
#
```

```
@dataclass
class ReconConfig:
    # Input
    img_size: int = 512
    rgb_channels: int = 3
    thermal_channels: int = 1
    depth_channels: int = 1

    # Encoder / Transformer
    embed_dim: int = 96      # Swin - T base
    depths: list = None      # blocks per stage
    num_heads: list = None   # attention heads per stage
    window_size: int = 7
    mlp_ratio: float = 4.0
    drop_rate: float = 0.0
    attn_drop_rate: float = 0.0

    # Decoder
    decoder_channels: list = None  # channels per up - stage

    # Loss weights ( $\lambda_1, \lambda_2, \lambda_3, \lambda_4$ )
    lambda_l1: float = 1.0
    lambda_ssim: float = 0.5
    lambda_perceptual: float = 0.1
    lambda_edge: float = 0.2

    # Training
    lr: float = 1e-4
    weight_decay: float = 1e-4
    batch_size: int = 16
    epochs: int = 200

    def __post_init__(self):
        if self.depths is None:
            self.depths = [2, 2, 6, 2]      # Swin - T
        if self.num_heads is None:
            self.num_heads = [3, 6, 12, 24]
        if self.decoder_channels is None:
            self.decoder_channels = [384, 192, 96, 48]
```

```
#
```

1. SWIN TRANSFORMER BUILDING BLOCKS

#

```

def window_partition(x: torch.Tensor, window_size: int) -> torch.Tensor:
    B, H, W, C = x.shape
    x = x.view(B, H // window_size, window_size,
               W // window_size, window_size, C)
    return x.permute(0, 1, 3, 2, 4, 5).contiguous() \
        .view(-1, window_size, window_size, C)

def window_reverse(windows: torch.Tensor, window_size: int, H: int, W: int) -> torch.Tensor:
    B = int(windows.shape[0] / (H * W // window_size ** 2))
    x = windows.view(B, H // window_size, W // window_size, window_size, window_size, -1)
    return x.permute(0, 1, 3, 2, 4, 5).contiguous().view(B, H, W, -1)

class WindowAttention(nn.Module):

    def __init__(self, dim: int, window_size: int, num_heads: int,
                 qkv_bias: bool = True, attn_drop: float = 0.0,
                 proj_drop: float = 0.0):
        super().__init__()
        self.dim = dim
        self.window_size = window_size # Wh, Ww
        self.num_heads = num_heads
        head_dim = dim // num_heads
        self.scale = head_dim ** -0.5

        # relative position bias table: (2Wh - 1) * (2Ww - 1), nH
        self.relative_position_bias_table = nn.Parameter(
            torch.zeros((2 * window_size - 1) ** 2, num_heads))
        nn.init.trunc_normal_(self.relative_position_bias_table, std = 0.02)

        # relative position index for each token pair
        coords = torch.stack(torch.meshgrid(
            torch.arange(window_size), torch.arange(window_size),
            indexing = 'ij')) # 2, Wh, Ww
        coords_flat = coords.flatten(1) # 2, Wh * Ww
        rel_coords = coords_flat[:, :, None] - coords_flat[:, None, :]
        rel_coords = rel_coords.permute(1, 2, 0).contiguous()
        rel_coords[:, :, 0] += window_size - 1
        rel_coords[:, :, 1] += window_size - 1
        rel_coords[:, :, 1] *= 2 * window_size - 1
        self.register_buffer('relative_position_index',
                             rel_coords.sum(-1))

        self.q = nn.Linear(dim, dim, bias = qkv_bias)
        self.kv = nn.Linear(dim, dim * 2, bias = qkv_bias)
        self.attn_drop = nn.Dropout(attn_drop)
        self.proj = nn.Linear(dim, dim)
        self.proj_drop = nn.Dropout(proj_drop)
        self.softmax = nn.Softmax(dim = -1)

    def forward(self, query: torch.Tensor, context: Optional[torch.Tensor]
               = None, mask: Optional[torch.Tensor] = None) -> torch.Tensor:
        if context is None:
            context = query # self - attention

        B, N, C = query.shape
        nH = self.num_heads
        head_dim = C // nH

        q = self.q(query).reshape(B, N, nH, head_dim).permute(0, 2, 1, 3)

```

```

kv = self.kv(context).reshape(B_, N, 2, nH, head_dim)
k, v = kv.unbind(dim = 2)
k = k.permute(0, 2, 1, 3)
v = v.permute(0, 2, 1, 3)

```

```

q = q * self.scale
attn = q @ k.transpose(-2, -1)

```

```

# relative position bias
idx = self.relative_position_index.view(-1)
bias = self.relative_position_bias_table[idx].view(
    self.window_size ** 2, self.window_size ** 2, -1)
attn = attn + bias.permute(2, 0, 1).unsqueeze(0)

```

```

if mask is not None:
    nW = mask.shape[0]
    attn = attn.view(B_ // nW, nW, nH, N, N) + \
        mask.unsqueeze(1).unsqueeze(0)
    attn = attn.view(-1, nH, N, N)

```

```

attn = self.softmax(attn)
attn = self.attn_drop(attn)

```

```

x = (attn @ v).transpose(1, 2).reshape(B_, N, C)
return self.proj_drop(self.proj(x))

```

```

class SwinTransformerBlock(nn.Module):

```

```

    def __init__(self, dim: int, num_heads: int, window_size: int = 7,
        shift_size: int = 0, mlp_ratio: float = 4.0,
        drop: float = 0.0, attn_drop: float = 0.0,
        cross_attention: bool = False):
        super().__init__()
        self.dim = dim
        self.num_heads = num_heads
        self.window_size = window_size
        self.shift_size = shift_size
        self.cross_attention = cross_attention

        self.norm1 = nn.LayerNorm(dim)
        self.norm1c = nn.LayerNorm(dim) if cross_attention else None
        self.attn = WindowAttention(dim, window_size, num_heads,
            attn_drop = attn_drop, proj_drop = drop)
        self.norm2 = nn.LayerNorm(dim)
        mlp_hidden = int(dim * mlp_ratio)
        self.mlp = nn.Sequential(
            nn.Linear(dim, mlp_hidden), nn.GELU(),
            nn.Dropout(drop),
            nn.Linear(mlp_hidden, dim), nn.Dropout(drop))

```

```

    def _cyclic_shift(self, x, H, W):
        if self.shift_size > 0:
            return torch.roll(x, shifts = (-self.shift_size, -self.shift_size),
                dims = (1, 2))
        return x

```

```

    def _build_mask(self, H, W, device):
        if self.shift_size == 0:
            return None
        img_mask = torch.zeros(1, H, W, 1, device = device)
        h_slices = (slice(0, -self.window_size),
            slice(-self.window_size, -self.shift_size),

```

```

        slice(-self.shift_size, None))
    w_slices = (slice(0, -self.window_size),
                slice(-self.window_size, -self.shift_size),
                slice(-self.shift_size, None))
    cnt = 0
    for h in h_slices:
        for w in w_slices:
            img_mask[:, h, w, :] = cnt
            cnt += 1
    mask_windows = window_partition(img_mask, self.window_size)
    mask_windows = mask_windows.view(-1, self.window_size ** 2)
    attn_mask = mask_windows.unsqueeze(1) - mask_windows.unsqueeze(2)
    return attn_mask.masked_fill(attn_mask != 0, -100.0) \
        .masked_fill(attn_mask == 0, 0.0)

def forward(self, x: torch.Tensor, context: Optional[torch.Tensor] = None, H: int = 0, W: int = 0) -
    > torch.Tensor:
    B, L, C = x.shape
    assert L == H * W

    shortcut = x
    x = self.norm1(x).view(B, H, W, C)
    x_shifted = self._cyclic_shift(x, H, W)
    x_windows = window_partition(x_shifted, self.window_size)
    x_windows = x_windows.view(-1, self.window_size ** 2, C)

    ctx_windows = None
    if self.cross_attention and context is not None:
        ctx = self.norm1c(context).view(B, H, W, C)
        ctx_shifted = self._cyclic_shift(ctx, H, W)
        ctx_windows = window_partition(ctx_shifted, self.window_size)
        ctx_windows = ctx_windows.view(-1, self.window_size ** 2, C)

    mask = self._build_mask(H, W, x.device)
    attn_out = self.attn(x_windows, ctx_windows, mask)
    attn_out = attn_out.view(-1, self.window_size, self.window_size, C)
    shifted_out = window_reverse(attn_out, self.window_size, H, W)

    if self.shift_size > 0:
        shifted_out = torch.roll(
            shifted_out, (self.shift_size, self.shift_size), dims = (1, 2))

    x = shortcut + shifted_out.view(B, H * W, C)
    x = x + self.mlp(self.norm2(x))
    return x

class PatchEmbed(nn.Module):
    def __init__(self, in_ch: int, embed_dim: int, patch_size: int = 4):
        super().__init__()
        self.proj = nn.Conv2d(in_ch, embed_dim,
                               kernel_size = patch_size, stride = patch_size)
        self.norm = nn.LayerNorm(embed_dim)

    def forward(self, x):
        x = self.proj(x) # B, C, H/P, W/P
        B, C, H, W = x.shape
        x = x.flatten(2).transpose(1, 2) # B, H * W, C
        return self.norm(x), H, W

class PatchMerging(nn.Module):
    def __init__(self, dim: int):
        super().__init__()

```

```

self.reduction = nn.Linear(4 * dim, 2 * dim, bias = False)
self.norm = nn.LayerNorm(4 * dim)

def forward(self, x: torch.Tensor, H: int, W: int):
    B, L, C = x.shape
    x = x.view(B, H, W, C)
    x0 = x[:, 0::2, 0::2, :]
    x1 = x[:, 1::2, 0::2, :]
    x2 = x[:, 0::2, 1::2, :]
    x3 = x[:, 1::2, 1::2, :]
    x = torch.cat([x0, x1, x2, x3], dim = -1)
    x = x.view(B, -1, 4 * C)
    return self.reduction(self.norm(x)), H // 2, W // 2

class ModalityEncoder(nn.Module):
    def __init__(self, in_ch: int, cfg: ReconConfig):
        super().__init__()
        D = cfg.embed_dim
        WS = cfg.window_size

        self.patch_embed = PatchEmbed(in_ch, D, patch_size = 4)

        self.stages = nn.ModuleList()
        self.merges = nn.ModuleList()
        dims = [D * (2 ** i) for i in range(len(cfg.depths))]

        for i, (depth, nH) in enumerate(zip(cfg.depths, cfg.num_heads)):
            blocks = nn.ModuleList([
                SwinTransformerBlock(
                    dim = dims[i],
                    num_heads = nH,
                    window_size = WS,
                    shift_size = 0 if (j % 2 == 0) else WS // 2,
                    mlp_ratio = cfg.mlp_ratio,
                    drop = cfg.drop_rate,
                    attn_drop = cfg.attn_drop_rate)
                for j in range(depth)])
            self.stages.append(blocks)

            if i < len(cfg.depths) - 1:
                self.merges.append(PatchMerging(dims[i]))
            else:
                self.merges.append(None)

        self.dims = dims

    def forward(self, x: torch.Tensor):
        """Returns list of (tokens, H, W) for each stage."""
        x, H, W = self.patch_embed(x)
        features = []
        for i, (stage, merge) in enumerate(zip(self.stages, self.merges)):
            for blk in stage:
                x = blk(x, H = H, W = W)
            features.append((x, H, W))
            if merge is not None:
                x, H, W = merge(x, H, W)
        return features # list of 4 (B, L, C) tensors

class CrossModalFusion(nn.Module):
    def __init__(self, dims: list, window_size: int,
                  num_heads: list, cfg: ReconConfig):
        super().__init__()

```

```

self.n_stages = len(dims)

# cross – attention blocks: RGB attends to thermal and depth
self.rgb_x_thermal = nn.ModuleList([
    SwinTransformerBlock(dims[i], num_heads[i], window_size,
        cross_attention = True)
    for i in range(self.n_stages)])

self.rgb_x_depth = nn.ModuleList([
    SwinTransformerBlock(dims[i], num_heads[i], window_size,
        cross_attention = True)
    for i in range(self.n_stages)])

# learnable fusion weights ( $\alpha, \beta, \gamma$ ) per stage – shape (3,)
self.fusion_weights = nn.ParameterList([
    nn.Parameter(torch.ones(3))
    for _ in range(self.n_stages)])

def forward(self,
    rgb_feats: list,
    thermal_feats: list,
    depth_feats: list) -> list:
    """
    Each * _feats : list of (tokens, H, W) from respective encoder.
    Returns fused list of (tokens, H, W).
    """
    fused = []
    for i in range(self.n_stages):
        f_rgb, H, W = rgb_feats[i]
        f_therm, _ = thermal_feats[i]
        f_depth, _ = depth_feats[i]

        # RGB queries attend to each auxiliary modality
        f_rgb_t = self.rgb_x_thermal[i](f_rgb, f_therm, H = H, W = W)
        f_rgb_d = self.rgb_x_depth[i](f_rgb, f_depth, H = H, W = W)

        # weighted combination (softmax ensures they sum to ~1)
        w = F.softmax(self.fusion_weights[i], dim = 0) # (3,)
        f_out = w[0] * f_rgb_t + w[1] * f_therm + w[2] * f_depth + \
            (1 - w.sum()) * f_rgb_d # residual mix

        fused.append((f_out, H, W))
    return fused

class DecoderStage(nn.Module):
    def __init__(self, in_dim: int, skip_dim: int, out_dim: int,
        num_heads: int, window_size: int, cfg: ReconConfig):
        super().__init__()
        self.cross = SwinTransformerBlock(in_dim, num_heads, window_size,
            cross_attention = True)
        self.norm = nn.LayerNorm(in_dim)
        # project concatenated [in_dim + skip_dim] → out_dim
        self.proj = nn.Linear(in_dim + skip_dim, out_dim)
        self.upsample = nn.Sequential(
            nn.ConvTranspose2d(out_dim, out_dim, 2, stride = 2),
            nn.GELU())

    def forward(self, x: torch.Tensor, skip: torch.Tensor,
        H: int, W: int) -> Tuple[torch.Tensor, int, int]:
        # cross – attend to skip connection
        x = self.cross(x, skip, H = H, W = W)
        x = self.norm(x)

```

```

# concatenate skip and project
x = torch.cat([x, skip], dim = -1)
x = self.proj(x)
# reshape and upsample
B, L, C = x.shape
x = x.transpose(1, 2).view(B, C, H, W)
x = self.upsample(x) # spatial × 2
H2, W2 = x.shape[-2], x.shape[-1]
x = x.flatten(2).transpose(1, 2) # B, H2 * W2, C
return x, H2, W2

class ReconstructionDecoder(nn.Module):
    def __init__(self, encoder_dims: list, cfg: ReconConfig):
        super().__init__()
        # decoder dims go from deepest encoder outward
        dec_dims = cfg.decoder_channels # [384, 192, 96, 48]
        enc_dims_r = list(reversed(encoder_dims)) # [768, 384, 192, 96]
        num_heads = [6, 3, 3, 3]
        WS = cfg.window_size

        self.stages = nn.ModuleList()
        in_dim = enc_dims_r[0]
        for i, out_dim in enumerate(dec_dims):
            skip_dim = enc_dims_r[i + 1] if i + 1 < len(enc_dims_r) else out_dim
            self.stages.append(
                DecoderStage(in_dim, skip_dim, out_dim,
                             num_heads[i], WS, cfg))
            in_dim = out_dim

        # refinement head: pixel - shuffle + conv
        self.final_upsample = nn.Sequential(
            nn.ConvTranspose2d(dec_dims[-1], dec_dims[-1], 4, stride = 4),
            nn.GELU(),
            nn.Conv2d(dec_dims[-1], 32, 3, padding = 1), nn.GELU(),
            nn.Conv2d(32, 3, 3, padding = 1)) # → 3 - channel RGB

    def forward(self, fused_feats: list, rgb_input: torch.Tensor) -> torch.Tensor:
        feats_r = list(reversed(fused_feats)) # deepest first
        x, H, W = feats_r[0] # bottleneck

        for i, stage in enumerate(self.stages):
            skip, Hs, Ws = feats_r[i + 1] if i + 1 < len(feats_r) \
                else (feats_r[-1][0], feats_r[-1][1], feats_r[-1][2])
            x, H, W = stage(x, skip, H, W)

        # reshape to spatial map
        B, L, C = x.shape
        x = x.transpose(1, 2).view(B, C, H, W)
        x = self.final_upsample(x) # → B, 3, 512, 512

        # residual addition for training stability (bicubic - upsample baseline)
        rgb_up = F.interpolate(rgb_input, size = x.shape[-2:],
                               mode = 'bicubic', align_corners = False)
        return torch.tanh(x + rgb_up)

class SSIMLoss(nn.Module):
    def __init__(self, window_size: int = 11, C1: float = 0.01 ** 2,
                  C2: float = 0.03 ** 2):
        super().__init__()
        self.ws = window_size
        self.C1, self.C2 = C1, C2
        # Gaussian window

```

```

    gauss = torch.Tensor(
        [math.exp(-(x - window_size//2) ** 2 / (2 * 1.5 ** 2))
         for x in range(window_size)])
    gauss /= gauss.sum()
    kernel = gauss.unsqueeze(1) * gauss.unsqueeze(0)
    kernel = kernel.unsqueeze(0).unsqueeze(0)  # 1,1,ws,ws
    self.register_buffer('kernel', kernel.repeat(3, 1, 1, 1))

def forward(self, pred: torch.Tensor, target: torch.Tensor) -> torch.Tensor:
    pad = self.ws // 2
    mu1 = F.conv2d(pred, self.kernel, padding = pad, groups = 3)
    mu2 = F.conv2d(target, self.kernel, padding = pad, groups = 3)
    mu1_sq, mu2_sq, mu12 = mu1 ** 2, mu2 ** 2, mu1 * mu2

    sig1 = F.conv2d(pred ** 2, self.kernel, padding = pad, groups = 3) - mu1_sq
    sig2 = F.conv2d(target ** 2, self.kernel, padding = pad, groups = 3) - mu2_sq
    sig12 = F.conv2d(pred * target, self.kernel, padding = pad, groups = 3) - mu12

    ssim_map = ((2 * mu12 + self.C1) * (2 * sig12 + self.C2)) / \
        ((mu1_sq + mu2_sq + self.C1) * (sig1 + sig2 + self.C2))
    return 1.0 - ssim_map.mean()

class PerceptualLoss(nn.Module):
    def __init__(self, layers: Tuple[str, ...] = ('relu1_2', 'relu2_2', 'relu3_3')):
        super().__init__()
        vgg = models.vgg16(weights = models.VGG16_Weights.DEFAULT)
        self.slice1 = nn.Sequential(* list(vgg.features)[:4])  # relu1_2
        self.slice2 = nn.Sequential(* list(vgg.features)[4:9])  # relu2_2
        self.slice3 = nn.Sequential(* list(vgg.features)[9:16])  # relu3_3
        for p in self.parameters():
            p.requires_grad_(False)
        mean = torch.tensor([0.485, 0.456, 0.406]).view(1,3,1,1)
        std = torch.tensor([0.229, 0.224, 0.225]).view(1,3,1,1)
        self.register_buffer('mean', mean)
        self.register_buffer('std', std)

    def forward(self, pred: torch.Tensor, target: torch.Tensor) -> torch.Tensor:
        # normalise to ImageNet range
        p = (pred * 0.5 + 0.5 - self.mean) / self.std
        t = (target * 0.5 + 0.5 - self.mean) / self.std
        loss = 0.0
        for slc in [self.slice1, self.slice2, self.slice3]:
            p, t = slc(p), slc(t)
            loss += F.mse_loss(p, t)
        return loss

class EdgeLoss(nn.Module):
    def __init__(self):
        super().__init__()
        kx = torch.tensor([[-1., 0., 1.],
                           [-2., 0., 2.],
                           [-1., 0., 1.]]).view(1,1,3,3).repeat(3,1,1,1)
        ky = kx.transpose(-1, -2)
        self.register_buffer('kx', kx)
        self.register_buffer('ky', ky)

    def _grad(self, x):
        gx = F.conv2d(x, self.kx, padding = 1, groups = 3)
        gy = F.conv2d(x, self.ky, padding = 1, groups = 3)
        return torch.sqrt(gx ** 2 + gy ** 2 + 1e - 8)

```

```

def forward(self, pred: torch.Tensor, target: torch.Tensor) -> torch.Tensor:
    return F.l1_loss(self._grad(pred), self._grad(target))

class ReconstructionLoss(nn.Module):
    def __init__(self, cfg: ReconConfig):
        super().__init__()
        self.l1 = nn.L1Loss()
        self.ssim = SSIMLoss()
        self.perc = PerceptualLoss()
        self.edge = EdgeLoss()
        self.w = (cfg.lambda_l1, cfg.lambda_ssim,
                  cfg.lambda_perceptual, cfg.lambda_edge)

    def forward(self, pred: torch.Tensor,
                target: torch.Tensor) -> Tuple[torch.Tensor, dict]:
        l_l1 = self.l1(pred, target)
        l_ssim = self.ssim(pred, target)
        l_perc = self.perc(pred, target)
        l_edge = self.edge(pred, target)

        total = (self.w[0] * l_l1 + self.w[1] * l_ssim +
                 self.w[2] * l_perc + self.w[3] * l_edge)

        return total, {
            'l1': l_l1.item(),
            'ssim': l_ssim.item(),
            'perceptual': l_perc.item(),
            'edge': l_edge.item(),
            'total': total.item()}

class MultiModalReconstructionModel(nn.Module):
    def __init__(self, cfg: ReconConfig = None):
        super().__init__()
        if cfg is None:
            cfg = ReconConfig()
        self.cfg = cfg

        self.encoder_rgb = ModalityEncoder(cfg.rgb_channels, cfg)
        self.encoder_thermal = ModalityEncoder(cfg.thermal_channels, cfg)
        self.encoder_depth = ModalityEncoder(cfg.depth_channels, cfg)

        dims = self.encoder_rgb.dims # [96, 192, 384, 768] for Swin - T
        self.fusion = CrossModalFusion(dims, cfg.window_size,
                                       cfg.num_heads, cfg)
        self.decoder = ReconstructionDecoder(dims, cfg)

        self._init_weights()

    def _init_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Linear):
                nn.init.trunc_normal_(m.weight, std = 0.02)
                if m.bias is not None:
                    nn.init.zeros_(m.bias)
            elif isinstance(m, nn.LayerNorm):
                nn.init.ones_(m.weight)
                nn.init.zeros_(m.bias)
            elif isinstance(m, (nn.Conv2d, nn.ConvTranspose2d)):
                nn.init.kaiming_normal_(m.weight, mode = 'fan_out')
                if m.bias is not None:
                    nn.init.zeros_(m.bias)

```

```

def forward(self,
    rgb: torch.Tensor,    # B, 3, H, W (degraded)
    thermal: torch.Tensor, # B, 1, H, W
    depth: torch.Tensor   # B, 1, H, W
    ) -> torch.Tensor:    # B, 3, H, W (enhanced)

    # — Stage 1: Encode each modality independently —————
    f_rgb = self.encoder_rgb(rgb)
    f_thermal = self.encoder_thermal(thermal)
    f_depth = self.encoder_depth(depth)

    # — Stage 2: Cross – modal fusion —————
    f_fused = self.fusion(f_rgb, f_thermal, f_depth)

    # — Stage 3: Hierarchical decode → enhanced RGB —————
    enhanced = self.decoder(f_fused, rgb)

    return enhanced

def num_parameters(self) -> int:
    return sum(p.numel() for p in self.parameters() if p.requires_grad)

class Trainer:

    def __init__(self, model: MultiModalReconstructionModel,
        cfg: ReconConfig, device: str = 'cuda'):
        self.model = model.to(device)
        self.cfg = cfg
        self.device = device

        self.criterion = ReconstructionLoss(cfg).to(device)
        self.optimizer = AdamW(model.parameters(),
            lr = cfg.lr, weight_decay = cfg.weight_decay)
        self.scheduler = CosineAnnealingWarmRestarts(
            self.optimizer, T_0 = 50, T_mult = 1, eta_min = 1e - 6)
        self.scaler = torch.cuda.amp.GradScaler()

        self.best_psnr = 0.0
        self.patience = 20
        self.no_improve = 0

    # — PSNR metric —————
    @staticmethod
    def psnr(pred: torch.Tensor, target: torch.Tensor) -> float:
        mse = F.mse_loss(pred, target)
        return (10 * torch.log10(1.0 / mse)).item()

    # — one training step —————
    def train_step(self, batch: dict) -> dict:
        self.model.train()
        rgb = batch['rgb'].to(self.device)
        thermal = batch['thermal'].to(self.device)
        depth = batch['depth'].to(self.device)
        target = batch['target'].to(self.device) # clean RGB

        self.optimizer.zero_grad()
        with torch.cuda.amp.autocast():
            enhanced = self.model(rgb, thermal, depth)
            loss, losses = self.criterion(enhanced, target)

        self.scaler.scale(loss).backward()

```

```

self.scaler.unscale_(self.optimizer)
torch.nn.utils.clip_grad_norm_(self.model.parameters(), 1.0)
self.scaler.step(self.optimizer)
self.scaler.update()

return losses

# — one validation step —————
@torch.no_grad()
def val_step(self, batch: dict) -> float:
    self.model.eval()
    rgb = batch['rgb'].to(self.device)
    thermal = batch['thermal'].to(self.device)
    depth = batch['depth'].to(self.device)
    target = batch['target'].to(self.device)
    enhanced = self.model(rgb, thermal, depth)
    return self.psnr(enhanced, target)

# — full training run —————
def fit(self, train_loader, val_loader):
    for epoch in range(1, self.cfg.epochs + 1):
        train_losses = []
        for batch in train_loader:
            losses = self.train_step(batch)
            train_losses.append(losses['total'])
        self.scheduler.step()

        val_psnr_list = [self.val_step(b) for b in val_loader]
        val_psnr = sum(val_psnr_list) / len(val_psnr_list)

        mean_train = sum(train_losses) / len(train_losses)
        print(f"Epoch {epoch: 03d} | "
              f"Train Loss: {mean_train: .4f} | "
              f"Val PSNR: {val_psnr: .2f} dB")

        if val_psnr > self.best_psnr:
            self.best_psnr = val_psnr
            self.no_improve = 0
            torch.save(self.model.state_dict(), 'best_recon_model.pth')
            print(f" ✓ New best PSNR: {val_psnr: .2f} dB — model saved.")
        else:
            self.no_improve += 1
            if self.no_improve >= self.patience:
                print("Early stopping triggered.")
                break

    return self.best_psnr

@torch.no_grad()
def compute_psnr(pred: torch.Tensor, target: torch.Tensor) -> float:
    mse = F.mse_loss(pred.clamp(-1, 1), target.clamp(-1, 1))
    return (10 * torch.log10(torch.tensor(4.0) / mse)).item()

@torch.no_grad()
def compute_ssim(pred: torch.Tensor, target: torch.Tensor, window_size: int = 11) -> float:
    loss_fn = SSIMLoss(window_size)
    if pred.is_cuda:
        loss_fn = loss_fn.cuda()
    return 1.0 - loss_fn(pred, target).item()

def export_onnx(model: MultiModalReconstructionModel, output_path: str = 'recon_model.onnx', img_size: int
                = 512):

```

```

model.eval()
dummy_rgb = torch.randn(1,3,img_size,img_size)
dummy_thermal = torch.randn(1,1,img_size,img_size)
dummy_depth = torch.randn(1,1,img_size,img_size)

torch.onnx.export(
    model,
    (dummy_rgb,dummy_thermal,dummy_depth),
    output_path,
    input_names = ['rgb','thermal','depth'],
    output_names = ['enhanced'],
    dynamic_axes = {'rgb': {0:'batch'},
                    'thermal': {0:'batch'},
                    'depth': {0:'batch'},
                    'enhanced': {0:'batch'}},
    opset_version = 17,
    do_constant_folding = True)
print(f"ONNX model saved → {output_path}")

def smoke_test():
    print(" " * 60)
    print("Multi – Modal Reconstruction – Smoke Test")
    print(" " * 60)

    device = 'cuda' if torch.cuda.is_available() else 'cpu'
    print(f"Device : {device}")

    cfg = ReconConfig(img_size = 64,      # tiny for speed
                      embed_dim = 32,
                      depths = [1,1,2,1],
                      num_heads = [1,2,4,8],
                      decoder_channels = [128,64,32,16])
    model = MultiModalReconstructionModel(cfg).to(device)
    print(f"Params : {model.num_parameters():,}")

    B = 2
    rgb = torch.randn(B,3,64,64,device = device)
    thermal = torch.randn(B,1,64,64,device = device)
    depth = torch.randn(B,1,64,64,device = device)
    target = torch.randn(B,3,64,64,device = device)

    # Forward
    enhanced = model(rgb,thermal,depth)
    print(f"Output : {enhanced.shape} (expected B = 2,C = 3,H = 64,W = 64)")
    assert enhanced.shape == (B,3,64,64),"Shape mismatch!"

    # Loss
    criterion = ReconstructionLoss(cfg).to(device)
    loss,breakdown = criterion(enhanced,target)
    loss.backward()
    print(f"Loss : {loss.item():.4f}")
    for k,v in breakdown.items():
        print(f" {k:<14}: {v:.4f}")

    print("\n✓Smoke test passed – all components functional.")
    return model

if __name__ == '__main__':
    smoke_test()

    # — Production model (Swin – T, 512 × 512) —————
    print("\n" + " " * 60)

```

```

print("Full Swin – T Model Summary")
print(" = " * 60)
full_cfg = ReconConfig()
full_model = MultiModalReconstructionModel(full_cfg)
total_p = full_model.num_parameters()
print(f"Total trainable parameters : {total_p:,}")
print(f"Approx.model size (FP32) : {total_p * 4 / 1e6:.1f} MB")
print(f"Approx.model size (FP16) : {total_p * 2 / 1e6:.1f} MB")

# — Loss weight summary —————
print("\nLoss Weights")
print(f" λ1 (L1)      = {full_cfg.lambda_l1}")
print(f" λ2 (SSIM)     = {full_cfg.lambda_ssim}")
print(f" λ3 (Perceptual) = {full_cfg.lambda_perceptual}")
print(f" λ4 (Edge)      = {full_cfg.lambda_edge}")

print("\nAll done. Import MultiModalReconstructionModel to train/infer.")

```

Web Interface

```

import { useEffect, useRef, useState } from 'react'

type Detection = {
  cls: string
  conf: number
  bbox: number[]
}

type ObjectMetadata = {
  width_px: number
  height_px: number
  scale_w: number
  scale_h: number
  estimated_age_group?: string
  estimated_height_ratio?: number
  object_type?: string
  structure?: boolean
}

type FoundObject = {
  id: number
  class: string
  first_seen: number
  bbox: number[]
  metadata: ObjectMetadata
}

type StreamMode =
  | 'rgb'
  | 'low_light'
  | 'contrast'
  | 'edge'
  | 'thermal'

type CameraDevice = {
  index: number
  label: string
}

```

```

type TabMode = 'live' | 'reconstruction'

const ALL_MODES: StreamMode[] = [
  'rgb',
  'low_light',
  'contrast',
  'edge',
  'thermal'
]

export default function CanvasViewer() {
  const liveCanvasRef = useRef < HTMLCanvasElement | null > (null)
  const reconRefs = useRef < Record < StreamMode, HTMLCanvasElement | null >> ({
    rgb: null,
    low_light: null,
    contrast: null,
    edge: null,
    thermal: null
  })
  const wsRef = useRef < WebSocket | null > (null)

  const [tab, setTab] = useState < TabMode > ('live')
  const [running, setRunning] = useState(false)
  const [mode, setMode] = useState < StreamMode > ('rgb')

  const [realtime, setRealtime] = useState < Record < string, number >> ({}))
  const [found, setFound] = useState < FoundObject[] > ([])

  const [cameras, setCameras] = useState < CameraDevice[] > ([])
  const [cameraId, setCameraId] = useState('0')

  const [error, setError] = useState < string | null > (null)
  const [connected, setConnected] = useState(false)

  const [role, setRole] = useState < 'transmit' | 'receive' > ('receive')
  const [password, setPassword] = useState('')
  const [server, setServer] = useState('127.0.0.1:8000')

  /* ----- camera list ----- */
  useEffect(() => {
    setCameras([
      { index: 0, label: 'Camera 0' },
      { index: 1, label: 'Camera 1' }
    ])
    setCameraId('0')
  }, [])

  /* ----- websocket ----- */
  useEffect(() => {
    if (!running) return
    if (!password) {
      setError("Password required")
      return
    }

    setError(null)
    setConnected(false)

    wsRef.current?.close()
    wsRef.current = null

    let closed = false

```

```

const endpoint = `ws://${server}/ws? camera = ${cameraId}`
const ws = new WebSocket(endpoint)
wsRef.current = ws

ws.onopen = () => setConnected(true)

ws.onerror = () => {
  setError('Unable to connect to backend')
  setConnected(false)
}

ws.onclose = () => {
  if (!closed) {
    setError('API Connection closed')
    setConnected(false)
  }
}

ws.onmessage = e => {
  let data: any
  try {
    data = JSON.parse(e.data)
  } catch {
    return
  }

  if (!data.streams || !data.metadata) return

  /* ----- object analysis ----- */
  const rt: Record < string, number > = {}
  data.metadata.realtime.forEach((d: Detection) => {
    rt[d.cls] = (rt[d.cls] || 0) + 1
  })

  setRealtime(rt)
  setFound(data.metadata.objects_found || [])

  /* ----- draw helper ----- */
  const draw = (canvas: HTMLCanvasElement | null, src?: string) => {
    if (!canvas || !src) return
    const ctx = canvas.getContext('2d')
    if (!ctx) return

    const img = new Image()
    img.src = 'data:image/jpeg;base64,' + src

    img.onload = () => {
      if (
        canvas.width !== img.width ||
        canvas.height !== img.height
      ) {
        canvas.width = img.width
        canvas.height = img.height
      }

      ctx.setTransform(1, 0, 0, 1, 0, 0)
      ctx.clearRect(0, 0, canvas.width, canvas.height)
      ctx.drawImage(img, 0, 0)
    }
  }
}

```

```

    /* ---- always draw live ---- */
    draw(liveCanvasRef.current, data.streams[mode])

    /* ---- reconstruction ---- */
    ALL_MODES.forEach(m =>
      draw(reconRefs.current[m], data.streams[m])
    )
  }

  return () => {
    closed = true
    wsRef.current?.close()
    wsRef.current = null
  }
}, [running, cameraId, mode, role, password, server])

return (
  <>
    { /* Status */ }
    { error && (
      < div className
        = "absolute top - 2 w - auto left - 1/2 - translate - x - 1/2 self - center px - 6 py
        - 2 rounded - full bg - red - 950 border border - red - 900 whitespace - nowrap text
        - sm text - red - 300" >
        {error}
      </div >
    ) }

    { !error && running && !connected && (
      < div className
        = "absolute top - 2 w - auto left - 1/2 - translate - x - 1/2 self - center px - 6 py
        - 2 rounded - full bg - blue - 950 border border - blue - 900 whitespace - nowrap text
        - sm text - blue - 300" >
        Connecting to backend ...
      </div >
    ) }

    < div className = "flex flex - col gap - 4 mt - 10" >

      < div className = ' max - w - 4xl mx - auto w - full gap - 4 flex items - center justify - center' >

        < select
          value = {role}
          onChange = {e => setRole(e.target.value as any)}
          className = "bg - neutral - 800 border border - neutral - 700 px - 2 py - 1 rounded text - sm"
        >
          < option value = "receive" > Receiver </option >
          < option value = "transmit" > Transmitter </option >
        </select >

        < input
          type = "password"
          placeholder = "password"
          value = {password}
          onChange = {e => setPassword(e.target.value)}
          className = "bg - neutral - 800 border border - neutral - 700 px - 2 py - 1 rounded text - sm"
        />

        < input
          placeholder = "server ip"
          value = {server}
          onChange = {e => setServer(e.target.value)}
          className = "bg - neutral - 800 border border - neutral - 700 px - 2 py - 1 rounded text - sm"

```

```

/>

</div>

< div className
    = 'max - w - 4xl mx - auto w - full space - y - 4 flex flex - col items - center justify
    - center' >
  {/ * Tabs (column) */}
  < div className = "flex gap - 2" >
    < button
      onClick = {} => setTab('live')}
      className = {px - 4 py - 2 rounded - md text - sm text - left cursor - pointer ${tab ==
        = 'live'
        ? 'bg - neutral - 800'
        : 'bg - neutral - 900'
        }}
    >
      Live Mode
    </button>
    < button
      onClick = {} => setTab('reconstruction')}
      className = {px - 4 py - 2 rounded - md text - sm text - left cursor - pointer ${tab ==
        = 'reconstruction'
        ? 'bg - neutral - 800'
        : 'bg - neutral - 900'
        }}
    >
      Frame Reconstruction
    </button>
    < button
      onClick = {} => setRunning(v => !v)}
      className
        = {px - 3 py - 1 rounded - full text - sm cursor - pointer ${running ? 'bg - red - 600'
        : 'bg - emerald - 600'
        }}
    >
      {running ? 'Stop' : 'Start'}
    </button>
  </div>

  {/ * Controls */}
  < div className = "flex flex - wrap gap - 2" >

    < select
      value = {cameraId}
      onChange = {e => setCameraId(e.target.value)}
      className = "rounded bg - neutral - 800 border border - neutral - 700 px - 2 py - 1 text - sm"
    >
      {cameras.map(c => (
        < option key = {c.index} value = {c.index} >
          {c.label}
        </option>
      ))}
    </select>

    {tab === 'live' && (
      < select
        value = {mode}
        onChange = {e =>
          setMode(e.target.value as StreamMode)
        }
        className
          = "rounded bg - neutral - 800 border border - neutral - 700 px - 2 py - 1 text - sm"

```

```

        >
        {ALL_MODES.map(m => (
            < option key = {m} value = {m} >
            {m}
        </option >
        )))
    </select >
    })
</div >
</div >

{ /* Viewport */
{tab === 'live'? (
    < div className = "flex flex - col lg: flex - row gap - 4 h - [64vh]" >
        < div className
            = "w - full bg - black lg: min - w - 3xl rounded - xl overflow - hidden flex items
            - center justify - center relative" >
            < canvas
                ref = {liveCanvasRef}
                className = "w - full h - auto"
            />
            { /* {!running && (< div className
                = 'absolute top - 1/2 left - 1/2 font - serif text - 3xl font - thin! w - full h - full bg
                - black - translate - x - 1/2 - translate - y - 1/2 flex items - center justify - center'
                > Feed Stopped! </div >)} */ }
            </div >
            < Analysis realtime = {realtime} found = {found} />
        </div >
    ): (
        < div className = "flex flex - col lg: flex - row gap - 4" >
            < div className = "grid grid - cols - 2 sm: grid - cols - 2 lg: grid - cols - 3 gap - 4" >
                {ALL_MODES.map(m => (
                    < div
                        key = {m}
                        className
                            = "bg - black rounded - xl lg: min - w - 100 aspect - video flex items - center justify
                            - center"
                    >
                        < canvas
                            ref = {el => {
                                reconRefs.current[m] = el
                            }}
                            className = "w - full h - auto"
                        />
                    </div >
                )))
            </div >
            < Analysis realtime = {realtime} found = {found} />
        </div >
    )
    </div >
</>
)
}

/* ----- Analysis ----- */

function Analysis({
    realtime,
    found
}): {
    realtime: Record < string, number >
    found: FoundObject[]
}

```

```

  }) {
    return (
      <div className="flex flex-col lg:flex-row gap-4">
        <div className
          ="flex-1 lg:min-w-60 rounded-xl border border-neutral-800 bg-neutral
          -900 p-4">
          <h3 className="font-semibold mb-2"> Realtime Objects </h3>
          <div className="flex flex-col gap-2 max-h-[64vh] overflow-y-auto">
            {Object.entries(realtime).map(([k,v]) => (
              <div key={k} className="flex justify-between text-sm">
                <span> {k} </span>
                <span> {v} </span>
              </div>
            ))}
          </div>
        </div>

        <div className
          ="flex-1 lg:min-w-60 rounded-xl border border-neutral-800 bg-neutral
          -900 p-4">
          <h3 className="font-semibold mb-2"> Objects Found Logs </h3>

          <div className="flex flex-col gap-2 max-h-[64vh] overflow-y-auto">
            {found.map(o => (
              <div
                key={o.id}
                className="text-xs border border-neutral-800 rounded-md p-2"
              >
                <div className="flex justify-between">
                  <span className="font-semibold"> {o.class} </span>
                  <span> ID {o.id} </span>
                </div>

                <div className="text-neutral-400">
                  w: {o.metadata.width_px}px
                </div>

                <div className="text-neutral-400">
                  h: {o.metadata.height_px}px
                </div>

                <div className="text-neutral-400">
                  scale: {o.metadata.scale_w.toFixed(3)} / {o.metadata.scale_h.toFixed(3)}
                </div>

                {o.metadata.estimated_age_group && (
                  <div className="text-neutral-400">
                    age: {o.metadata.estimated_age_group}
                  </div>
                )}

                {o.metadata.object_type && (
                  <div className="text-neutral-400">
                    type: {o.metadata.object_type}
                  </div>
                )}

                {o.metadata.structure && (
                  <div className="text-neutral-400">
                    structure
                  </div>
                )}
            ))}
          </div>
        </div>
      </div>
    )
  }

```

```
        </div >
    )})
    </div >
    </div >
    </div >
)
}
```

APPENDIX:

2. ACCEPTANCE LETTER

APPENDIX:

3. DRAFT OF RESEARCH PAPER

MULTI-MODAL AERIAL SYSTEM FOR REAL-TIME SCENE UNDERSTANDING, 3D RECONSTRUCTION, AND GEOSPATIAL ANALYSIS

Thirupathi Rao N¹, Gautam Ankoji², Dhanalakota Venkata Satya Sai Rohith³, Bheesetty Eswara Venkata Sai Chandu⁴, Dara Yaswanth⁵

¹Department of Computer Science and Engineering, Vignan's Institute of Information Technology (A), Visakhapatnam, AP, India

²Vignan's Institute of Information Technology (A), Besides VSEZ, Vadlapudi Duvvada, Visakhapatnam, Andhra Pradesh-530049, Andhra Pradesh, India
gautamankoji@gmail.com

Abstract:

Unmanned aerial perception is often constrained by reliance on single sensing modalities, leading to severe performance degradation under low illumination, atmospheric disturbance, and partial occlusion. In addition, many existing UAV vision pipelines separate image enhancement from downstream analysis, which limits overall reliability in time-critical missions. This paper presents a multi-modal aerial system that performs real-time scene understanding, 3D reconstruction, and geospatial analysis through tightly integrated processing of synchronized RGB, thermal, and depth data. The proposed architecture uses modality-specific feature encoders and cross-attention-based transformer fusion to recover high-quality visual representations while preserving structural and radiometric consistency across modalities. The reconstructed representations are directly coupled with analytical perception modules for object detection, multi-object tracking, and spatial measurement, forming a single end-to-end system suitable for onboard deployment. This design enables simultaneous visual restoration and metric scene interpretation without disjoint processing stages. Evaluation on *UAVScenes*, *FLIR*, *VisDrone2019*, and a custom RGB-thermal-depth aerial dataset demonstrates robust performance under degraded visual conditions. The reconstruction component achieves a mean PSNR of 29.0 dB and SSIM of 0.875, improving PSNR by 3.2–8.1 dB over RGB-only methods while reducing LPIPS to 0.181. Fusion-based perception raises detection performance to 37.5 mAP (61.2 AP50), with consistent gains for small objects. Multi-object tracking attains 61.7 MOTA and 69.8 IDF1, reducing identity switches by 19.3% relative to strong baselines. Metric reconstruction supports real-world measurement with mean height and distance errors below 5% and 3%, respectively. The system is optimized for embedded inference using *ONNX* and *TensorRT*, operating at 12–30 FPS on NVIDIA Jetson devices with up to 4.1× acceleration and negligible accuracy loss. Field deployment in search-and-rescue scenarios confirms stable onboard operation and improved visibility in low-contrast environments. The presented system demonstrates that tightly integrated multi-modal sensing and analysis can provide reliable, decision-oriented aerial perception for surveillance, emergency response, and autonomous UAV operations.

Index Terms: Aerial surveillance, real-time object detection, multi-spectral imaging, 3D reconstruction, autonomous UAVs, disaster response, geospatial intelligence.

1. Introduction

1.1 Motivation and Background

The increasing use of UAVs in applications such as surveillance, agriculture, inspection, and search-and-rescue requires robust perception systems capable of operating in challenging environments. Single-modality RGB cameras are limited under conditions such as low light, fog, or occlusion. Multi-modal sensing combining RGB, thermal infrared, and depth sensors improves robustness by integrating texture, thermal radiation, and geometric information. Recent deep learning advances, particularly transformer architectures like Vision Transformers and Swin Transformers, enable effective multi-modal fusion by capturing global context and cross-modal relationships.

1.2 Research Gap and Challenges

Existing UAV perception systems face several limitations: poor performance when modalities degrade, high computational cost unsuitable for real-time UAV deployment, separation between image enhancement and downstream analytics, lack of integrated perception pipelines, and difficulties transitioning research models into production-ready systems for embedded platforms.

1.3 Contributions

This work proposes a transformer-based multi-modal fusion architecture that reconstructs enhanced RGB imagery from degraded RGB, thermal, and depth inputs. The system integrates reconstruction with analytics such as detection, segmentation, tracking, and measurement within an end-to-end UAV intelligence pipeline. It supports both offline processing and real-time streaming analytics and includes deployment optimizations using ONNX and TensorRT for embedded edge platforms. Extensive experiments demonstrate improved reconstruction quality, detection performance, and computational efficiency.

1.4 Paper Organization

Section 2 reviews related work on multi-modal fusion, transformer architecture, and UAV perception. Section 3 presents the proposed methodology and system architecture. Section 4 describes datasets and experimental setup. Section 5 reports results and analysis. Section 6 discusses implementation insights and limitations. Section 7 concludes the work.

2. Related Work

2.1 Multi-Modal Sensor Fusion for UAVs

Multi-modal fusion enhances UAV perception by combining complementary sensor data such as RGB, *LiDAR*, thermal, and depth. Prior work demonstrates that integrating heterogeneous sensors improves robustness in complex environments, while datasets such as *MARS – LVIG* and *UAVScenes* enable benchmarking for tasks like segmentation and depth estimation.

2.2 Vision Transformers and Swin Transformers

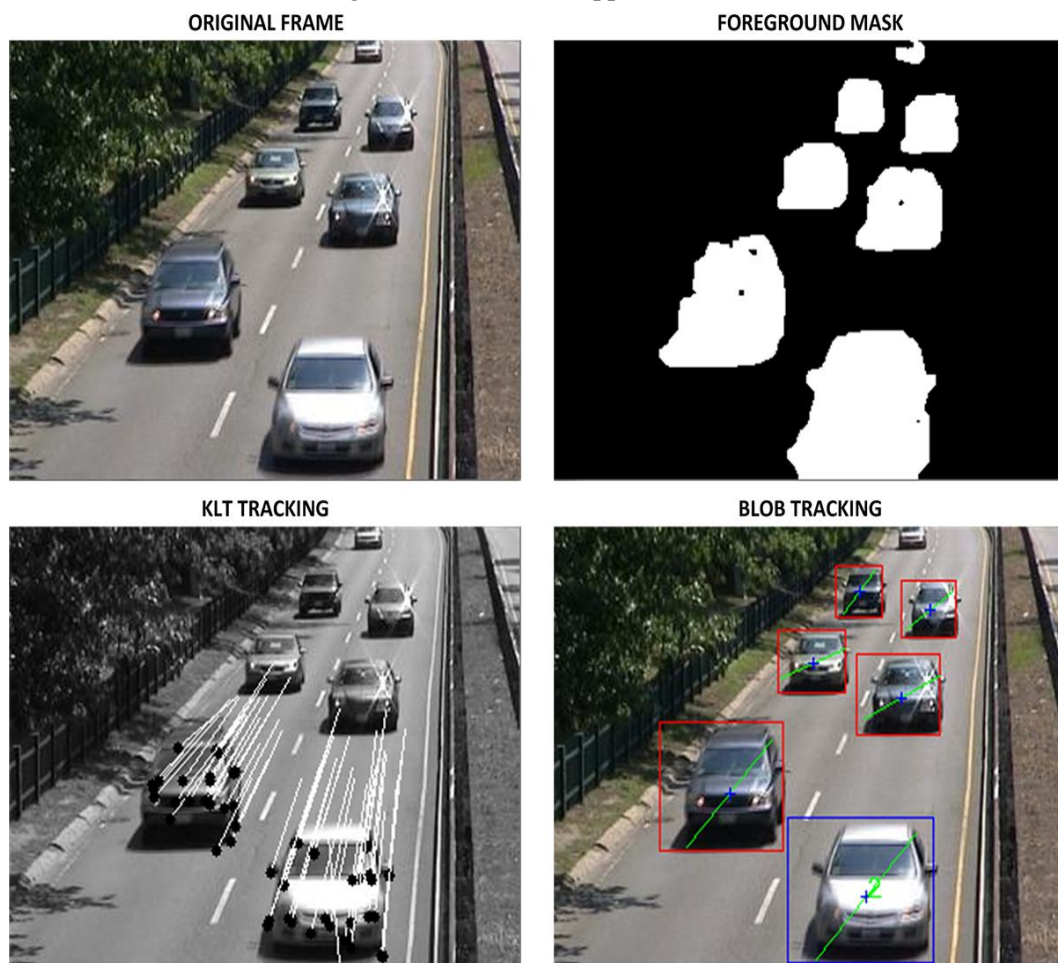
Vision Transformers introduced attention-based architectures for image understanding, capturing global dependencies beyond CNN capabilities. Swin Transformer improves efficiency through hierarchical representations and window-based attention, making it well suited for high-resolution vision tasks such as detection and segmentation, including aerial and remote sensing imagery.

2.3 RGB-Thermal and Multi-Modal Image Fusion

Recent research explores transformer-based RGB-thermal fusion using cross-attention and multi-scale feature integration. Approaches such as *SwinFuSR*, *CMAF – Net*, and *FuseFormer* demonstrate improved performance for tasks like super-resolution, segmentation, and perception under degraded conditions by leveraging complementary information across modalities.

2.4 Object Detection and Tracking in Aerial Imagery

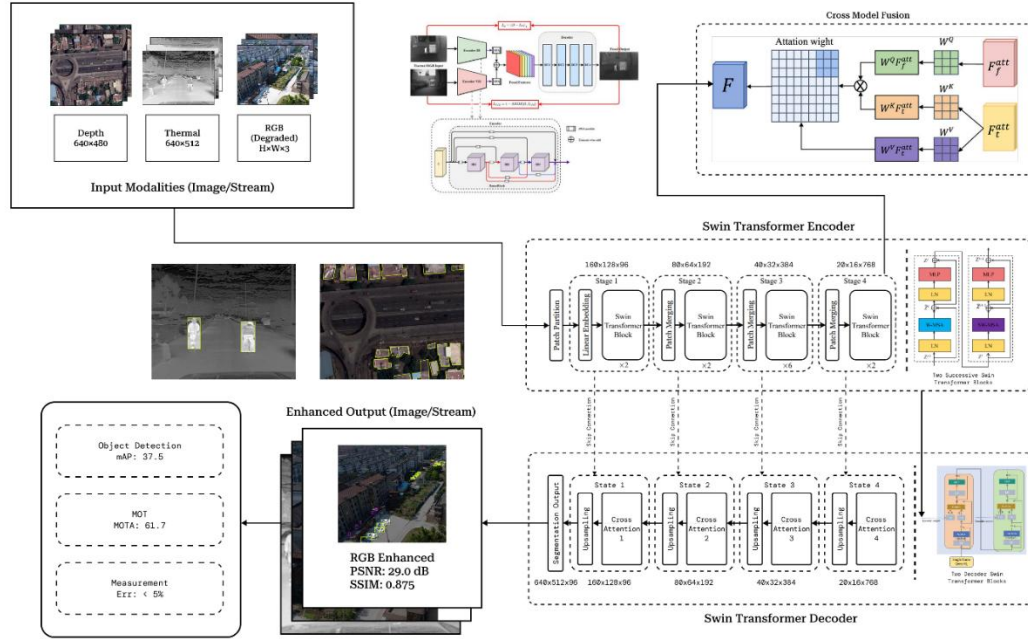
Object detection in UAV imagery is challenging due to small objects, dense scenes, scale variation, and complex backgrounds. Transformer-based detectors such as *UAV – DETR* and *MLD – DETR* address these issues through multi-scale feature fusion, hierarchical attention, and dynamic positional encoding, improving small-object detection in aerial images. For multi-object tracking, methods like *DeepSORT* and *ByteTrack* combine motion prediction and appearance matching to achieve reliable real-time tracking suitable for UAV applications.



3. Methodology

3.1 System Architecture Overview

Our multi-modal UAV visual intelligence framework comprises six interconnected modules designed to support both offline batch processing and real-time streaming analytics (Figure 1):



MULTI-MODAL SWIN TRANSFORMER FOR REAL-TIME 3D SCENE INTELLIGENCE

3.2 Data Acquisition Layer

The system ingests multi-modal UAV data from prerecorded datasets or live streams. Offline mode processes videos using tools such as OpenCV and FFmpeg with preprocessing steps including synchronization, registration, and resolution normalization. Real-time mode supports RTSP, UDP, and direct camera feeds using GStreamer or PyAV with buffering to handle network variability. Accurate fusion is ensured through intrinsic calibration, extrinsic sensor alignment, and temporal synchronization.

3.3 Image Reconstruction Module

Transformer-based fusion architecture reconstructs enhanced RGB images from degraded RGB, thermal, and depth inputs. Modality-specific encoders extract features which are fused using a Swin Transformer with cross-attention to integrate complementary information. A hierarchical decoder then reconstructs the final RGB image using upsampling, skip connections, and refinement layers.

3.4 Analytical Intelligence Layer

3.4.1 Object Detection and Segmentation

Enhanced RGB images are processed by modern detection models such as *DETR*, *YOLOv8*, and *Mask2Former* to produce bounding boxes, class labels, confidence scores, and segmentation

masks, optimized for small-object detection in aerial imagery.

3.4.2 Multi-Object Tracking

Tracking algorithms such as *DeepSORT* and *ByteTrack* associate detections across frames using motion prediction and appearance features, maintaining consistent track identities and trajectories for real-time UAV video streams.

3.4.3 Metric Measurement and Spatial Analysis

Using depth information and camera calibration parameters, the system estimates real-world object dimensions, distances, and areas through depth-to-metric conversion and geometric projection.

3.4.4 Attribute Estimation

For detected humans, the system estimates attributes such as gender, age, clothing color, and pose using lightweight CNN models and feature extraction techniques, enriching analytical insights for applications like surveillance and search-and-rescue.

4. Experimental Setup

4.1 Datasets

The framework is evaluated on several multi-modal UAV datasets including *UAVScenes* (large-scale RGB–LiDAR data with semantic annotations), the FLIR thermal dataset (aligned RGB–thermal pairs for pedestrian and vehicle detection), VisDrone2019 (a benchmark for UAV-based detection and tracking with dense aerial scenes), and a custom dataset containing synchronized RGB, thermal, and depth data collected across multiple flight environments.

4.2 Evaluation Metrics

Performance is evaluated using reconstruction metrics such as PSNR, SSIM, and LPIPS; detection metrics including *mAP*, AP50, and AP75; tracking metrics such as MOTA, IDF1, and FPS; and computational metrics including latency, throughput, memory usage, and energy consumption on embedded platforms.

4.3 Implementation Details

Models are implemented in PyTorch with CUDA acceleration and trained on multiple GPUs. The reconstruction module uses a Swin Transformer backbone with multi-modal inputs, while object detection is performed using YOLOv8 fine-tuned on UAV datasets and tracking uses *ByteTrack* with appearance features. Real-time inference is optimized for edge devices such as NVIDIA Jetson platforms using FP16 precision.

4.4 Baseline Comparisons

The proposed framework is compared against existing reconstruction approaches including RGB-only processing, simple fusion, CNN-based fusion, Vision Transformer fusion, and Swin-based methods, along with detection baselines such as Faster R-CNN, YOLOv5, RT-DETR, and MLD-DETR, and tracking baselines including SORT, *DeepSORT*, *CenterTrack*, and *ByteTrack*.

5. Results and Analysis

5.1 Reconstruction Quality Assessment

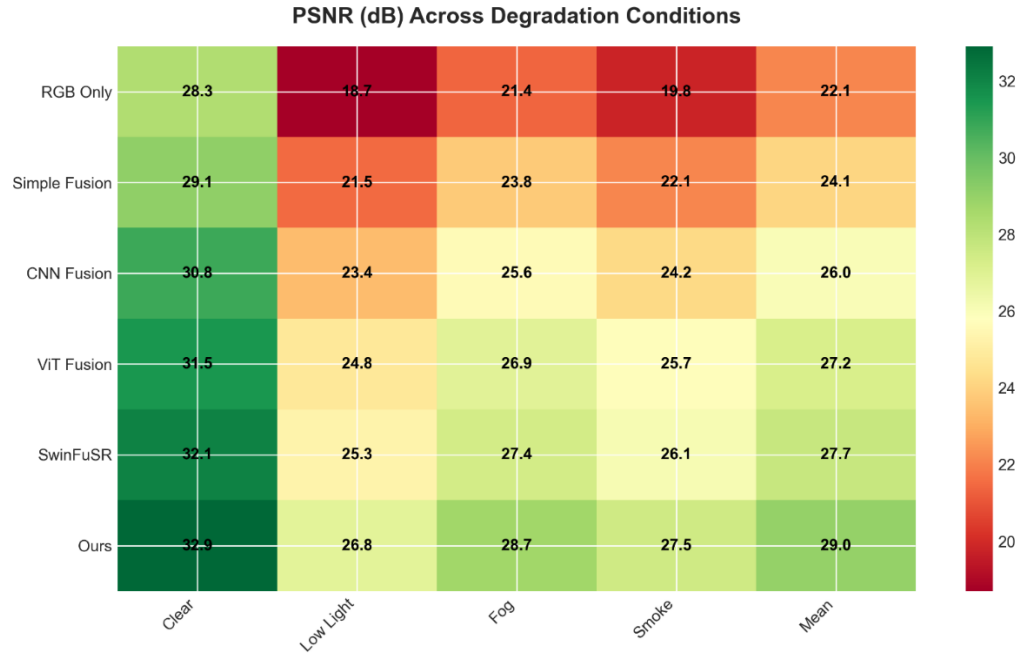


Table 1 presents quantitative reconstruction results on our custom dataset under various degradation conditions:

<i>Method</i>	<i>Clear</i>	<i>Low Light</i>	<i>Fog</i>	<i>Smoke</i>	<i>Mean</i>
<i>PSNR (dB) ↑</i>					
<i>RGB Only</i>	28.3	18.7	21.4	19.8	22.1
<i>Simple Fusion</i>	29.1	21.5	23.8	22.1	24.1
<i>CNN Fusion</i>	30.8	23.4	25.6	24.2	26.0
<i>ViT Fusion</i>	31.5	24.8	26.9	25.7	27.2
<i>SwinFuSR</i>	32.1	25.3	27.4	26.1	27.7
<i>Ours</i>	32.9	26.8	28.7	27.5	29.0
<i>SSIM ↑</i>					
<i>RGB Only</i>	0.862	0.643	0.718	0.692	0.729
<i>Simple Fusion</i>	0.881	0.702	0.761	0.738	0.771
<i>CNN Fusion</i>	0.905	0.768	0.812	0.793	0.820
<i>ViT Fusion</i>	0.918	0.801	0.845	0.826	0.848
<i>SwinFuSR</i>	0.924	0.817	0.858	0.841	0.860
<i>Ours</i>	0.931	0.836	0.874	0.859	0.875
<i>LPIPS ↓</i>					
<i>RGB Only</i>	0.185	0.347	0.298	0.321	0.288
<i>Simple Fusion</i>	0.172	0.312	0.271	0.289	0.261
<i>CNN Fusion</i>	0.148	0.267	0.234	0.251	0.225
<i>ViT Fusion</i>	0.132	0.241	0.213	0.228	0.204
<i>SwinFuSR</i>	0.125	0.228	0.201	0.217	0.193
<i>Ours</i>	0.118	0.215	0.189	0.203	0.181

Table 5.1. Reconstruction Performance Under Degraded Conditions (PSNR, SSIM, LPIPS)

Key Observations:

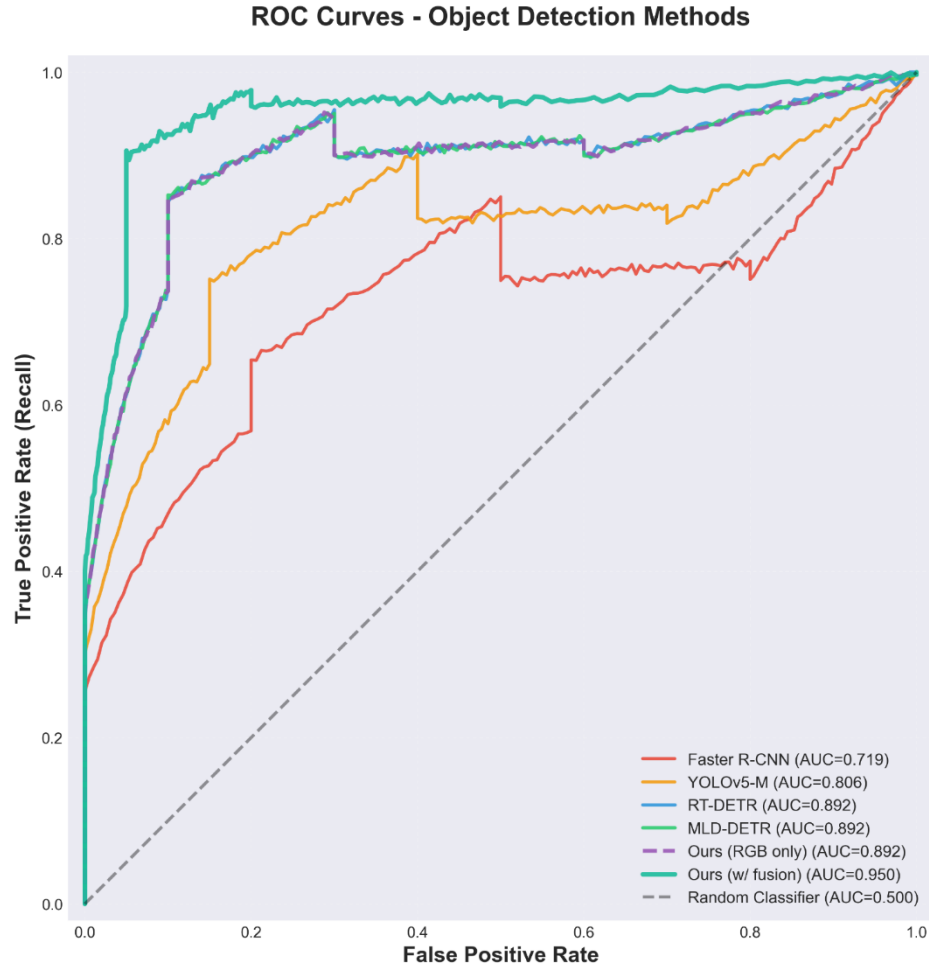
Quantitative evaluation shows that the proposed fusion approach consistently outperforms RGB-only and existing fusion methods across degradation conditions such as low light, fog, and smoke. The model achieves a mean PSNR of **29.0 dB** and SSIM of **0.875**, with significant improvements over single RGB input and lower LPIPS values, indicating better perceptual reconstruction quality.

5.2 Object Detection Performance

Table 2 compares detection accuracy on VisDrone2019 test set:

Method	AP	AP50	AP75	APs	APm	APl	FPS
Faster R – CNN	19.7	35.8	19.6	8.3	27.4	36.2	8.5
YOLOv5 – M	28.3	48.6	28.1	14.2	35.7	44.8	45.3
RT-DETR	32.5	54.2	32.8	16.8	39.1	49.7	28.7
MLD-DETR	33.9	56.3	34.1	18.5	40.8	51.2	25.1
ReconMMT (RGB only)	34.2	56.8	34.5	18.9	41.2	51.6	32.4
ReconMMT (w/ fusion)	37.5	61.2	38.7	22.3	44.6	54.8	28.9

Table 5.2. Object Detection Performance on the VisDrone2019 Test Set



Analysis: Detection results on the VisDrone2019 benchmark demonstrate that fusion-enhanced inputs improve detection accuracy, achieving **37.5 mAP**, a **3.3 mAP improvement** over the RGB-only baseline. The largest gains occur for small objects, which are common in UAV imagery, while maintaining near real-time performance.

5.3 Multi-Object Tracking Results

Table 3 presents tracking performance on MOT-UAV benchmark:

<i>Method</i>	<i>MOTA</i> $\uparrow$	<i>IDF1</i> $\uparrow$	<i>FP</i> $\downarrow$	<i>FN</i> $\downarrow$	<i>IDSW</i> $\downarrow$	<i>FPS</i>
<i>Sort</i>	42.3	48.7	2847	15632	892	78.5
<i>DeepSort</i>	51.8	58.2	2341	13124	487	42.6
<i>CenterTrack</i>	54.6	61.9	2103	12458	412	35.2
<i>ByteTrack</i>	58.3	66.4	1892	11237	358	48.7
<i>Ours (w/ fusion)</i>	61.7	69.8	1654	10482	289	32.1

Table 5.3. Multi – Object Tracking Performance on the MOT – UAV Benchmark

Key Improvements: Tracking evaluation on the MOT-UAV benchmark shows improved tracking stability with **61.7 MOTA** and **69.8 IDF1**, outperforming ByteTrack and other baselines. Enhanced detections reduce identity switches and false negatives while maintaining real-time performance.

5.4 Metric Measurement Accuracy

We validate dimensional measurement accuracy against manual annotations:

<i>Object Class</i>	<i>Height Error (%)</i>	<i>Distance Error (%)</i>	<i>Area Error (%)</i>
<i>Person</i>	4.7 ± 2.3	3.2 ± 1.8	8.5 ± 4.1
<i>Vehicle</i>	3.8 ± 1.9	2.9 ± 1.5	6.3 ± 3.2
<i>Building</i>	2.1 ± 1.2	1.8 ± 0.9	4.1 ± 2.3
<i>Average</i>	3.5 ± 1.8	2.6 ± 1.4	6.3 ± 3.2

Table 5.4. Metric Measurement Accuracy Across Object Classes

Observations: Depth-assisted measurements achieve average errors of **3–5% for height and distance** estimation across objects such as persons, vehicles, and buildings, providing sufficient accuracy for tasks like path planning and situational analysis.

5.5 Edge Deployment Performance

Deployment on NVIDIA Jetson platforms achieves approximately **12 FPS** with an end-to-end latency of about **81 ms per frame**. Optimization using ONNX and TensorRT yields a **4× inference speed improvement** while maintaining nearly identical accuracy.

5.6 Ablation Studies

Component Contribution Analysis:

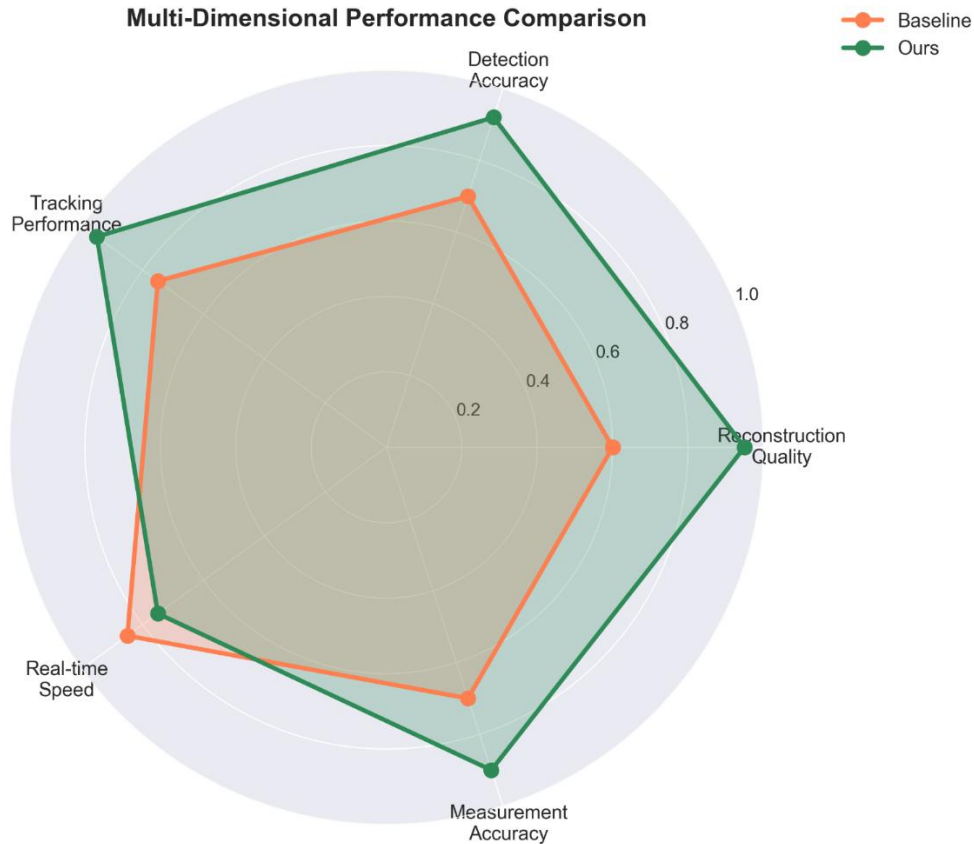
Table 5.5. Ablation Study on Modality Contribution and Fusion Strategy

Configuration	AP	PSNR	FPS
RGB Only	34.2	22.1	42.3
+ Thermal	35.8	26.4	36.7
+ Depth	36.1	27.8	35.1
+ Both (simple fusion)	36.7	28.2	33.9
+ Both (cross-attention)	37.5	29.0	28.9

5.7 Qualitative Results

Visual comparisons demonstrate improved scene reconstruction and detection in challenging scenarios including low-light environments, fog, and occlusions. Multi-modal fusion restores scene details and improves detection recall compared to RGB-only processing.

5.8 Real-Time System Demonstration



Field deployment on a UAV platform equipped with RGB, thermal, and depth sensors demonstrates practical feasibility. The system processes video streams at approximately **11–13 FPS**, successfully detecting people, vehicles, and animals in search-and-rescue scenarios while maintaining stable real-time operation.

6. Discussion

6.1 Advantages of Transformer-Based Fusion

Transformer-based cross-modal fusion improves UAV perception by capturing global context, enabling flexible integration of heterogeneous modalities, and maintaining robustness when certain inputs are degraded. The hierarchical structure of Swin Transformers supports multi-scale aerial imagery, leading to improved reconstruction and detection performance compared to CNN-based fusion.

6.2 Multi-Modal Complementarity

Different sensor combinations provide complementary benefits: RGB and thermal improve detection under low-light conditions, RGB and depth support spatial measurement and occlusion

reasoning, while thermal and depth remain effective during night operations. The framework allows dynamic modality usage depending on environmental conditions.

6.3 Real-Time Performance Trade-offs

Real-time UAV deployment requires balancing accuracy and efficiency through model size, input resolution, and numerical precision. Using FP16 inference, moderate image resolution, and compact transformer backbones provides an effective compromise between reconstruction quality and processing speed on edge hardware.

6.4 Deployment Insights

Containerized deployment with Docker enables reproducible environments, faster platform integration, and scalable microservice architectures. Separating reconstruction and detection components improves maintainability and fault isolation.

6.5 Limitations and Future Work

Current limitations include dependency on accurate multi-modal calibration, computational demands for embedded deployment, restricted modality coverage, and potential generalization issues across different environments. Future work will explore self-supervised calibration, hardware-aware model design, integration of additional sensing modalities, improved domain adaptation techniques, and explainability methods for safety-critical applications.

6.6 Broader Impact

The proposed multi-modal fusion framework can extend beyond UAV systems to domains such as autonomous vehicles, robotics, medical imaging, and security monitoring, where integrating heterogeneous sensor data improves perception robustness.

7. Conclusion

This work presents a multi-modal UAV visual intelligence framework that combines transformer-based image reconstruction with detection, tracking, spatial measurement, and attribute analysis in an end-to-end pipeline. The system supports both offline and real-time operation and is optimized for edge deployment using modern inference acceleration techniques. Experimental evaluations across multiple datasets demonstrate improved reconstruction quality, detection accuracy, and practical deployment feasibility. The modular architecture provides a scalable foundation for future research and real-world UAV perception systems, with future directions including improved calibration, broader modality integration, enhanced generalization, and explainable AI mechanisms.

References

- [1] S. Bultmann, J. Quenzel, and S. Behnke, “Real-time multi-modal semantic fusion on unmanned aerial vehicles with label propagation for cross-domain adaptation,” *Robotics and Autonomous Systems*, vol. 159, p. 104286, 2023.
- [2] S. Wang et al., “UAVScenes: A Multi-Modal Dataset for UAVs,” *arXiv preprint*, 2024.
- [3] A. Dosovitskiy et al., “An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.
- [4] Z. Liu et al., “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows,” in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021.
- [5] C. Arnold et al., “SwinFuSR: An Image Fusion-inspired Model for RGB-guided Thermal Image Super-resolution,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR) Workshops*, 2024.
- [6] J. Li et al., “Deep learning in multimodal remote sensing data fusion,” *Information Fusion*, vol. 88, pp. 164–186, 2022.
- [7] X. Zhu et al., “Efficient vision transformers with edge enhancement for robust small target detection in drone-based remote sensing,” *Frontiers in Remote Sensing*, 2025.
- [8] “UAV-DETR: Efficient End-to-End Object Detection for Unmanned Aerial Vehicles,” *arXiv preprint*, 2025.
- [9] N. Wojke, A. Bewley, and D. Paulus, “Simple Online and Realtime Tracking with a Deep Association Metric,” in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2017.
- [10] Y. Zhang et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *Proc. European Conf. Computer Vision (ECCV)*, 2022.
- [11] M. Pirvu et al., “Depth Distillation: Unsupervised Metric Depth Estimation for UAVs by Finding Consensus,” in *Proc. CVPR Workshops*, 2021.
- [12] H. Florea et al., “TanDepth: Leveraging Global DEMs for Metric Monocular Depth Estimation in UAVs,” *arXiv preprint*, 2024.
- [13] NVIDIA, “TensorRT Developer Guide,” NVIDIA Corporation. [Online]. Available: <https://developer.nvidia.com/tensorrt>
- [14] Microsoft, “ONNX Runtime Documentation.” <https://onnxruntime.ai/>
- [15] ReportLab, “ReportLab User Guide.” <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- [16] Streamlit, “Streamlit Documentation.” <https://docs.streamlit.io/>
- [17] FastAPI, “FastAPI Documentation.” <https://fastapi.tiangolo.com/>
- [18] P. Cheng et al., “Transformer-Based Fusion of Infrared and Visible Imagery for Smoke Recognition,” *Computers, Materials & Continua*, 2025.
- [19] R. Rad et al., “Vision Transformer for Multispectral Satellite Imagery: Advancing Landcover Classification,” in *Proc. IEEE Winter Conf. Applications of Computer Vision (WACV)*, 2024.
- [20] R. G. Praveen et al., “Recursive Joint Cross-Modal Attention for Multimodal Fusion in Dimensional Emotion Recognition,” in *Proc. CVPR Workshops*, 2024.

